

RELAY — Deadline-Aware Food Rescue Dispatch

Master Report · AmiHacks Track A, Problem Statement 1: "Surplus-to-Shelter: Real-Time Food Rescue Routing"

What this document is: the single source of truth for the project. It covers the problem, the research, what we concluded from it, the options we considered, the design, the math, data and simulation, evaluation, the 24-hour build plan, the demo, the pitch, the risks and a final verdict.

Prepared: 24 Sep 2026 · **Format:** Markdown with Mermaid and LaTeX math (renders on GitHub, Obsidian and VS Code with the Markdown Preview Mermaid Support extension)

How to read this document: epistemic labels

The brief asked us to keep facts, research, ideas and guesses clearly separate. Every non-obvious claim carries one of these tags:

Tag	Meaning
[PS]	Stated in the official AmiHacks problem statement PDF
[R]	Taken from an external source we checked; the citation number points to §References
[R-2nd]	External claim we saw only second-hand, in another paper's description. Treat with care
[INF]	Our inference from the research. It is reasoned but has not been tested in the field
[ASM]	An assumption we are making, which must be validated or made configurable
[PROP]	Our proposal or design decision
[UNV]	Commonly cited but not verified by us against a primary source

Every performance number this project produces during the hackathon comes from simulation and must be presented that way. Nothing in this report is a real-world result from our system.

Table of Contents

- [0. Executive Summary](#)
- [1. Understanding the Problem Statement](#)
- [2. Research: The Real-World Problem](#)
- [3. The Real Problem Underneath](#)
- [4. Solution Directions Explored](#)
- [5. Final Solution: Relay](#)
- [6. AI / ML / RL: A Critical Evaluation](#)
- [7. Product Thinking](#)
- [8. Technical Design](#)
- [9. Mathematical Formulation](#)
- [10. Data & Simulation](#)

- [11. Evaluation Framework](#)
 - [12. 24-Hour Hackathon Strategy](#)
 - [13. Demo Strategy](#)
 - [14. The Judges' Perspective](#)
 - [15. Failure & Adversarial Analysis](#)
 - [16. Business & Real-World Deployment](#)
 - [17. Implementation Blueprint](#)
 - [18. Pitch & Demo](#)
 - [19. Final Critical Verdict](#)
 - [Appendix A: Safety rules table \(conservative defaults; must be validated locally\). \[R5, ASM\]](#)
 - [Appendix B: Planner defaults \(tune in simulation\).](#)
 - [Appendix C: Assumptions register](#)
 - [References](#)
-

0. Executive Summary

The stated problem [PS]: match surplus food from restaurants, grocers and caterers to shelters in real time, dispatch a volunteer, track status and report impact. The PS says the food has a 2–6 hour usable window.

What we think the real problem is [INF, grounded in R]: finding a recipient is the easy part. Copia, a commercial platform, reports a 99% match rate for surplus it handles [R4]. What goes wrong is **fulfilment under a hard deadline, using a workforce you cannot command**. Volunteers are *offered* work. They are not *dispatched*. The best public data on this comes from 412 Food Rescue in Pittsburgh: across 4,574 rescues, **749 (16.4%) were never claimed or were only claimed in the final hour**, which the authors attribute to a dispatcher phoning regular volunteers [R1]. The notification rule behind those numbers was fixed for every rescue: notify within 5 miles, notify everyone after 15 minutes, dispatcher steps in during the final hour [R1]. Two further difficulties sit on top. Notifications cost something, because they burn out volunteers [R1, R2]. And the obvious fixes worsen geographic inequity: a volunteer recommender reportedly reached about 90% completion downtown and about 40% in outer suburbs [R-2nd, R3].

So the real problem is **reliability engineering for a perishable, time-windowed pickup-and-delivery system run by probabilistic human labour**. Three clocks have to line up:

1. the **food-safety clock**: the FDA Food Code's time-as-a-public-health-control rule allows 4 hours, or 6 hours for cold food kept $\leq 70^{\circ}\text{F}$ [R5];
2. the **donor's window**: they close and want the food gone;
3. the **recipient's receiving window and capacity**: is the shelter open, and does it have space?

What we propose to build [PROP]: Relay, a dispatch co-pilot for food-rescue coordinators. It gives every donation:

- a **safety deadline**, set by rules and never by ML;
- a **committed plan and a backup plan**;
- a **live probability that the donation will be claimed in time**, from a small, calibrated acceptance model;
- **offer waves sized to each donation's own deadline** (412FR used one fixed rule for all);
- an explicit **latest safe escalation time**: the last moment a human or paid backup can still save the food. The dispatcher is alerted *before* that point, not in the final hour;
- **event-triggered re-planning** when a volunteer cancels, a shelter fills up or a surge arrives, with multi-stop insertion offered to volunteers who are already on the road.

Donors post in plain language over a chat channel, and an LLM does **extraction only**, never safety decisions. Recipients manage capacity with one tap. Every state change goes into an **append-only event log**, which serves both as the audit trail (FSSAI's surplus-food rules require records [R6]) and as the replay engine for the simulator.

The demo: the same random seed runs through the same simulated Friday night twice, side by side. On one side is a WhatsApp-style broadcast baseline plus a 412-style static rule. On the other is Relay. A volunteer cancels, a shelter fills and a caterer posts a 25 kg surge. The audience watches the baseline lose food to the safety clock while Relay escalates early and re-routes. Results across 30 seeds are shown with confidence intervals, and the demo says plainly that they are simulated.

What we deliberately do *not* build [PROP]:

- **Reinforcement learning.** It isn't justified. In the one published food-rescue notification benchmark we found, a DQN policy scored below a simple greedy policy [R2].
- An **agentic LLM negotiator**.
- **Computer-vision quantity estimation**.
- **Demand forecasting** without data.

1. Understanding the Problem Statement

1.1 Extraction from the official PDF [PS]

Element	What the PS says
Track	Track A · NGO / Social Impact · "Surplus-to-Shelter: Real-Time Food Rescue Routing"
Tagline	"Turn a restaurant's unsold food into a shelter's next meal — before it hits the dumpster."
Core problem	No fast, reliable way to connect surplus that is <i>available right now, nearby</i> with <i>who can pick it up and use it before it spoils</i> . Coordination runs on phone calls, spreadsheets and WhatsApp groups, and "breaks down at scale and under time pressure".
Time window	"most surplus food has only a 2–6 hour usable window"
Objective	Donor posts in under a minute → system automatically matches to the best-fit recipient on capacity, need and distance → coordinates pickup → captures data for measurable diversion impact . The "how" is left open.
Stakeholders	Primary: donors (restaurants, grocers, caterers, campus dining), rescue orgs/shelters, volunteer or gig drivers. Secondary: local government/health departments, corporate ESG teams, food-security researchers.
Pain points	Manual coordination; no view of shelter capacity ("can they even take 40 lbs of pasta tonight?"); no systematic tracking for impact or tax; food expires before a match; a trust gap .
Why it is hard (PS's own words)	"real-time matching under time constraints, geographic routing optimization, trust/verification between parties who do not know each other, and handling messy, inconsistent data about food type, quantity, and safety windows."
Required capabilities	(1) fast intake; (2) real-time matching; (3) driver dispatch or routing suggestion; (4) recipient capacity and preference management; (5) status tracking from posted to matched to picked up to delivered; (6) impact dashboard (meals, weight, CO2e); (7) notifications by SMS, email or push; (8) <i>optional</i> food-safety or expiry-risk scoring.
Innovation menu	Forecasting to pre-position volunteers; an agentic dispatcher; CV intake; NLP intake; multi-stop routing; analytics and hotspot maps; cloud geo-matching.

Element	What the PS says
Explicit guidance	"Only use what is genuinely useful — a well-executed matching engine beats a bolted-on AI feature."
Constraints	Safety ("never route expired-risk food"); privacy; reliability ("a failed match wastes the food anyway"); accessibility for non-technical staff and drivers; scale beyond one city; near-instant matching; ethics ("avoid creating friction that discourages small donors").
24h scope	MVP: donor posts → system matches (rule-based or ML) → dispatch view → dashboard. Advanced: CV intake, multi-stop routing, agentic dispatcher.

1.2 Implicit challenges the PS points at but doesn't name [INF]

1. **Drivers are volunteers, not employees.** "Dispatch" really means asking someone who can say no, fail to reply, or accept and then cancel. Classical vehicle routing assumes you control the fleet. Here you don't, and that changes the model.
2. **"Best-fit" has more than one objective.** Distance, capacity, need and fairness pull against each other. The nearest shelter being always chosen starves shelters further away.
3. **Every match needs three windows to overlap.** Donor availability, food-safety deadline and recipient receiving hours must all intersect. The PS mentions expiry but not the recipient's hours. [INF] Restaurant surplus appears at closing time, often late evening, when many recipients may not be able to receive. This is a hypothesis to check with local NGOs.
4. **"Near-instant" is a product requirement.** The donor should get a *credible* answer within seconds: "someone is coming by 10:40" or "we can't make it tonight."
5. **Trust runs both ways.** Donors worry about liability and whether food is actually used. Recipients worry about safety and being used as a dump. Volunteers worry about wasted trips.
6. **Impact numbers get audited.** ESG and tax use means the method must be defensible and conservative.

1.3 What the PS does *not* ask for

- A consumer marketplace that *sells* surplus (the Too Good To Go model). That is a different problem.
- Household food waste. It is the largest share of food waste globally (60% [R7]) but is outside this PS.
- A warehouse or food-bank inventory system. The PS is direct transfer from donor to recipient.

2. Research: The Real-World Problem

2.1 Scale and why it matters

Fact	Source
1.05 billion tonnes of food waste in 2022, including inedible parts, which is 132 kg/capita and almost one-fifth of food available to consumers. Households 60%, food service 28%, retail 12%.	[R7] UNEP Food Waste Index 2024 press release
Food loss and waste produce 8–10% of global GHG emissions. 783 million people were affected by hunger.	[R7]
India: about 55 kg/capita/year household food waste and about 78.2 Mt total, per WRI India citing UNEP FWI 2024. Caveat: India's estimate rests on seven sub-national studies from 2014–2018, and food service and retail data for India is "lacking."	[R8] (secondary; primary Annex 3 not read by us)

Fact	Source
US: 29% of the 240 Mt food supply went unsold or uneaten in 2024, per ReFED.	[R9]
US EPA: about 58% of fugitive methane emissions from municipal solid waste landfills come from landfilled food waste, and food is about 24% of landfilled MSW.	[R10]
California SB 1383: recover 20% of edible food that would otherwise be landfilled by 2025. Regulated donors must hold written agreements with recovery organisations and keep records , including pounds per month per organisation.	[R11]

What this means for us [INF]. The food-service and retail share, which is what this PS targets, is a *minority* of total waste, and only part of it is safely rescuable edible surplus. Our impact claims must stay modest. The Indian data gap on food-service waste also means **there is no public dataset to train a forecasting model on**. That limits how much ML we can use (see §6).

2.2 How food rescue actually works today

Organisation / product	Model (as publicly described)	What we take from it
412 Food Rescue / Food Rescue Hero (Pittsburgh; platform licensed to other orgs) [R1, R12]	Volunteers get app alerts for pickups, see food, quantity and distance, and use their own cars. There is no warehouse. The FRH site lists about 20+ partner orgs, over 63,000 registered drivers, and "one million food rescues" celebrated in Sept 2026. The documented notification rule (2018–19): push to volunteers within 5 miles → after 15 minutes push to everyone → in the last hour the dispatcher phones regulars [R1].	This is the leading <i>volunteer-dispatch</i> platform. Its main operational pain, documented in its own research, is unclaimed rescues and dispatcher load .
Food Rescue US (39 locations, 22 states + DC) [R13]	Web app. Donors register food, agencies post needs, volunteers self-schedule. Volunteers are encouraged to " adopt " recurring rescues. Each location has a Site Director who vets donors. An average rescue takes about 30 minutes. Donors track their own tax records.	Mature operations shift volume into recurring, scheduled rescues . Ad-hoc real-time rescue is the hard tail.
Copia (US PBC, all 50 states, Canada, PR) [R4]	B2B. Pickup logistics, tax receipts, tax, ESG and SB 1383 compliance dashboards. Donors pay a percentage of donation value plus an optional subscription. Nonprofits use it free. Claims a "99% match rate."	Businesses will pay when compliance or tax value is clear. Matching is essentially solved.
Too Good To Go (21 countries, 500M+ meals, 180k business partners) [R14]	Sells surplus "Surprise Bags" to consumers at about one-third of price.	Different problem. It shows that surplus which can be <i>sold</i> often is, so donated surplus is what's left over. [INF]
OLIO (9.3M users, 100k+ trained volunteers) [R15]	Peer-to-peer sharing plus "Food Waste Heroes" volunteers who collect from businesses; partner logos include Tesco and Pret.	Volunteer collection works at scale when the task is predictable and routine.

Organisation / product	Model (as publicly described)	What we take from it
Robin Hood Army (India, founded 2014, "zero-funds") [R16]	Decentralised volunteer chapters, mostly students and young professionals. Reports 406 cities and 158M meals across 2020–24. Contact and joining run through WhatsApp.	The main Indian alternative is a process, not software. Coordination runs through chapter WhatsApp groups [INF from the site's WhatsApp-first contact; ops details not published]. Any Indian product must meet people on WhatsApp.
Feeding India (Eternal/Zomato) [R17]	The current site describes <i>funded</i> daily meals for children (1.4 lakh+ children a day) and does not mention surplus rescue.	A large Indian player's public programme is now funded meals rather than surplus rescue. We don't know why and won't speculate.
FSSAI "Save Food Share Food" + IFSA [R6, R18]	Regulator-led programme since 2018. The Indian Food Sharing Alliance is a network of 85 surplus-food distribution NGOs. An IVR helpline and app were planned. The Recovery & Distribution of Surplus Food Regulations 2019 require hygiene compliance, suitable transport, storage and reheating, and "a proper record of surplus food". Donors must ensure food "is not unsafe".	Record-keeping is a regulatory need in India , so our event log is required rather than a nice-to-have. IFSA is a natural distribution partner.

2.3 Legal and safety frame

- **Food-safety clock [R5]:** Under the FDA Food Code §3-501.19 (Time as a Public Health Control), food removed from temperature control at $\leq 41^{\circ}\text{F}$ (5°C) or $\geq 135^{\circ}\text{F}$ (57°C) must be discarded at **4 hours**. Cold food can go **6 hours** if it never exceeds 70°F (21°C). The time it left temperature control must be marked. This lines up with the PS's "2–6 hour" window. [ASM] FSSAI's regulations, as far as we could verify, do not publish an equivalent hour limit. **We adopt the FDA rule as a conservative, configurable default** and flag it for validation by the local food-safety authority.
- **Liability:** In the US, the Bill Emerson Act protects good-faith donors, and the **Food Donation Improvement Act of 2022** extended protection to *direct* donations by "qualified direct donors" (restaurants, caterers, grocers and others) to needy individuals at zero cost [R19]. In India, FSSAI's 2019 regulations aim to protect good-faith donors but place safety obligations on donors and distribution organisations [R6]. [UNV] The exact Indian liability protections should be checked by counsel.
- **Incentives:** US tax deductions exist, including enhanced deductions (Copia explains the PATH Act rule [R4]). In India, the relevant institutional lever is **CSR**: Companies Act 2013 Schedule VII item (i) lists "eradicating hunger, poverty and malnutrition" as an eligible CSR activity [R20]. [UNV] Tax treatment of in-kind food donations in India was not verified; don't claim it in the pitch.

2.4 Academic research: what's known

Work	Contribution	What it means for us
Shi, Yuan, Lo, Lizarondo, Fang, IAAI-20 [R1]	412FR data from Mar 2018 to May 2019: 4,574 rescues, 749 "negative" (672 never claimed, plus 77 claimed only in the last hour by regulars). They predict whether a rescue will be claimed and optimise a static Intervention &	This validates our framing that claim reliability is the bottleneck. Their scheme is <i>static</i> and optimised over the

Work	Contribution	What it means for us
	Notification Scheme (x = second-wave time, y = first-wave radius, z = intervention lead time). The stacking model reached AUC 0.81, but gradient boosting reached only AUC 0.51 , so the problem is hard. The optimised scheme B was projected to cut interventions by 24% (about 390 a year). No field results reported.	whole history. Our angle is making it dynamic and per-donation, driven by each donation's own deadline.
Raman, Shi, Fang, NeurIPS 2024 (RMAB with global rewards) [R2]	Arms are volunteers, pulls are notifications, and the reward is $P(\text{at least one accepts}) = 1 - \prod(1 - a_i p_i)$. The notification budget exists because "too many notifications can lead to burned-out volunteers." On the notification task, Linear-Whittle scored 1.93 and greedy 1.83, above DQN at 1.55 (normalised to random).	Our wave-sizing formula is this same noisy-OR. The evidence does not favour deep RL here.
Tang, Raman, Fang, Shi, 2025 preprint (Contextual Budget Bandits) [R3]	Existing engagement algorithms make geographic disparities worse . The paper cites a recommender with roughly 90% completion downtown versus about 40% in suburbs [R-2nd]. It proposes region-specific notification budgets.	We add an equity guard : deadline-driven waves automatically spend more notifications on low-supply areas, and we measure per-zone rescue rate.
RescueLens (Raman et al., AI Magazine 2026 / arXiv 2025) [R21]	An LLM triages volunteer feedback. It is deployed at 412FR, recovers 96% of issues at 71% precision, and found 0.5% of donors responsible for over 30% of volunteer issues.	The right use of LLMs in this domain is structuring messy text, not deciding.
Nair, Rey, Dixit, IISE Transactions 2017 [R22]	Fair allocation and cost-effective routing models for food rescue and redistribution.	Fairness and routing are coupled. We use a two-stage approach (allocate, then route) with a fairness term.
Lien, Iravani, Smilowitz, Operations Research 2014 [R23]	Sequential allocation for nonprofits that maximises the expected minimum fill rate , balancing equity against waste.	This is the basis for our recipient-priority term: favour recipients with low fill rate against need.
Mittal, Gibson, Krejci (agent-based crowd-shipping for food rescue, WSC 2019; IJPDLM 2021) [R24]	Simulation study of crowd-shipped food rescue.	There is precedent for agent-based simulation as the evaluation method in this domain.
Alonso-Mora et al., PNAS 2017 [R25]	Ride-sharing: batch requests over roughly 30 s, then an anytime ILP assigning trips to vehicles, plus rebalancing. About 3M NYC trips, real time.	Batched, rolling-horizon, anytime optimisation is the proven pattern for dynamic dispatch.
Xu et al. (DiDi), KDD 2018 [R26]	Learned spatiotemporal values combined with combinatorial dispatch at DiDi scale.	Learning-plus-planning pays off at millions of trips a day

Work	Contribution	What it means for us
		with rich logs. That is not our regime.

2.5 Algorithms and tooling (verified)

- **Google OR-Tools** (Apache-2.0) [R27] supports pickup-and-delivery pairs (`AddPickupAndDelivery`), time windows via a time dimension, **optional visits via** `AddDisjunction(nodes, penalty)`, soft upper bounds on cumulative variables (for tardiness), guided local search and a hard `time_limit`.
- **VROOM** (BSD-2) [R28] handles PDPTW "shipments" with time windows, capacities, skills and priority 0–100, returns unassigned tasks, and runs on OSRM, ORS or Valhalla matrices.
- **OSRM** (BSD-2) [R29]: the public demo server allows at most 1 request/second and non-commercial use only. **Precompute and cache matrices.**
- **Li & Lim PDPTW benchmarks** [R30]: 354 instances of 100–1000 tasks, useful for sanity-checking a solver but not for our domain.

2.6 Public data for simulation

- **Not public, as far as we found:** real food-rescue operational logs. The 412FR data used in [R1–R3] came from a partner. The RMAB code is public on GitHub [R2]; we did not verify whether raw data is included, so **assume it is not**.
- **Usable:** OpenStreetMap points of interest via the Overpass API (restaurants, `amenity=social_facility`, shelters and similar) give a realistic *spatial* layout of donors and recipients. OSRM gives road travel times (cache them). Everything *temporal* (arrival rates, acceptance behaviour) is **synthetic and parameterised** (§10).

3. The Real Problem Underneath

3.1 Five "why"s

1. *Why is food wasted when people need it?* It isn't moved in time.
2. *Why not?* Nobody reliably commits to moving it before the deadline.
3. *Why not?* The movers are volunteers deciding in the moment. Broadcasts get ignored ("someone else will take it"), offers go to the wrong people, and nobody knows a rescue is failing until it is too late.
4. *Why doesn't anyone know?* No system tracks the **slack**, meaning how much time remains before rescue becomes impossible, and so human intervention is triggered by a fixed rule ("last hour") rather than by risk.
5. *Why is that hard?* Slack depends on three windows, travel times, recipient capacity and uncertain human responses, and it changes with every event.

3.2 Diagnosis

Candidate "core problem"	Verdict	Reasoning
Matching donation → recipient	Secondary	Cheap to compute and rarely the binding constraint (a 99% match rate at Copia [R4]). It still matters for capacity, fairness and dietary fit.
Routing (multi-stop)	Secondary	Volunteers mostly do single rescues in their own cars, averaging about 30 minutes [R13]. Multi-stop helps with large or surge events and paid vans.

Candidate "core problem"	Verdict	Reasoning
Reliability and timing of human commitment	PRIMARY	16.4% of rescues needed rescuing themselves [R1]. Static rules waste notifications on easy rescues and start too late on hard ones.
Coordination under uncertainty (re-planning when commitments break)	PRIMARY	"A failed match wastes the food anyway" [PS]. Cancellations and full shelters are exactly the events that break WhatsApp coordination.
Recipient capacity visibility	Important	Named explicitly in the PS. A delivery the recipient can't use just moves waste downstream. ReFED's method assumes 6.5% of rescued food is still wasted [R9].
Incentives and trust	Important, mostly product and ops	Handled through audit trail, proof of delivery, receipts and honest ETAs.
Prediction of supply	Premature	No data. The best "forecast" is converting frequent donors into scheduled rescues [INF from R13].

Problem statement, restated [PROP]:

Given perishable donations with hard safety deadlines, recipients with time-varying capacity and hours, and a volunteer workforce that can only be offered work, maximise the safe, usable food delivered, while (a) minimising volunteer notification burden, (b) triggering human or paid backup early enough to matter, and (c) spreading service fairly across recipients and neighbourhoods.

3.3 Trade-offs baked into the problem

- **Notify more** → faster claims but volunteer burnout [R1, R2]. **Notify less** → more unclaimed rescues.
- **Nearest recipient** → less driving but unfairness. **Most-needy recipient** → longer trips and a tighter safety margin.
- **Re-optimising aggressively** → better plans, but confusing churn for humans who were already told a plan.
- **Strict safety defaults** → some edible food rejected. **Loose defaults** → health risk. The PS says never route expired-risk food, so strict wins.

3.4 Secondary harms our solution could cause [INF]

1. **Waste displacement:** food is dumped on shelters that can't use it. *Mitigation:* capacity holds, category acceptance and a "could you use this?" confirmation for large drops.
2. **Volunteer burnout** from notification spam. *Mitigation:* a per-volunteer weekly offer budget and minimal wave sizing.
3. **Geographic inequity**, where downtown gets served and the periphery doesn't [R3]. *Mitigation:* deadline-driven escalation spends effort where risk is highest, per-zone metrics, and a fairness term.
4. **Moral hazard:** "donation as free disposal" encourages over-production or donors pushing marginal-quality food. *Mitigation:* donor reliability scores (RescueLens found 0.5% of donors cause over 30% of issues [R21]) and condition checks at pickup.
5. **Safety incident** that destroys trust for the whole network. *Mitigation:* hard deadline rules, a checklist at pickup, a "not for human consumption" diversion path, and the audit log.
6. **Privacy harm:** exposing the addresses of sensitive recipients such as women's shelters. *Mitigation:* approximate location until acceptance, and a proxy handover point for confidential sites.

4. Solution Directions Explored

We looked at ten directions. We scored each one 1 (weak) to 5 (strong) on the brief's six criteria. The scores are our judgement [INF].

4.1 Candidates

A. Rule-based nearest-match + broadcast (the "WhatsApp group in an app" approach)

- *Idea*: The donor posts, the nearest recipient that has capacity is chosen, and every volunteer is notified. The first one to claim it wins.
- *Solves*: Basic digitisation, and it creates a status trail.
- *Weaknesses*: It has no idea whether a rescue is failing. It floods volunteers with notifications and never re-plans. Most hackathon teams will build this.
- *Difficulty*: Low. *24h*: Easy. **We build it anyway, as the baseline.**

B. Batched optimal assignment (a min-cost matching of donations to recipients and volunteers, re-run on a rolling horizon)

- *Idea*: Every Δ seconds, solve an assignment problem over the open donations, recipients and available volunteers.
- *Weaknesses*: It treats volunteers as if they could be dispatched. In reality they have to be *offered* the job and can say no. An "optimal" assignment that the volunteer declines is worthless.
- *Difficulty*: Medium.

C. Dynamic PDPTW routing (multi-stop, time windows, capacities, done in OR-Tools)

- *Idea*: Classic dynamic vehicle routing with pickup–delivery pairs and time windows.
- *Weaknesses*: The same fleet-control assumption as B. It is overkill for rescues where one volunteer does one pickup, and the solver takes time to learn.
- *Strength*: It really does help for surges, large caterer loads, and volunteers who are already carrying food and can pick up more on the way.
- *Difficulty*: Medium–high.

D. Reliability layer (acceptance probability, deadline-aware offer waves, latest-safe-escalation, and backup plans)

- *Idea*: Model how each volunteer is likely to respond, then size notification waves to each donation's own slack. Compute when escalation to a human or paid backup must happen, and alert *before* that point.
- *Solves*: The primary bottleneck in §3.
- *Weaknesses*: The acceptance model has no real data during the hackathon. It needs a simulator to show it works.
- *Novelty*: It extends the 412FR *static* scheme [R1] to a *per-donation, dynamic* one. It uses the same noisy-OR objective as [R2], but with deadline-driven escalation. We know of no public tool that does this. We cannot see inside proprietary platforms, though, so we don't claim it is unique.
- *Difficulty*: Medium.

E. Agentic LLM dispatcher (an LLM negotiates pickup times with donor, recipient and driver)

- *Weaknesses*: Adds latency, can't be audited, and can hallucinate commitments in a safety-critical flow. A time-slot picker does the same job deterministically.
- *Novelty*: Looks novel, but it adds little over the deterministic version.

F. RL dispatch/notification policy

- *Weaknesses*: There is no real environment data, so a policy trained on our simulator just learns our simulator's assumptions. In the most relevant published benchmark, DQN did worse than greedy [R2]. It is also hard to explain to a dispatcher. See §6.

G. Supply forecasting to pre-position volunteers (a PS suggestion)

- *Weaknesses:* No historical data exists. India's food-service waste data is "lacking" [R8]. At our scale the simplest effective version is **spotting recurring donors and proposing a scheduled rescue**, which is counting, not ML.

H. CV intake (photo → food type and quantity)

- *Weaknesses:* Estimating quantity from a photo is unreliable, and errors spill into capacity decisions. It is at most a pre-fill suggestion that the donor has to confirm.

I. Conversational intake (free text or chat → structured donation via an LLM, with a confirmation step)

- *Solves:* The PS's "post in under a minute" and "messy data" requirements. It meets Indian donors where they already are, on WhatsApp.
- *Weaknesses:* LLMs can extract things wrong. So there is a confirmation card, a strict schema, and a regex fallback.
- *Difficulty:* Low–medium.

J. Surplus marketplace / sell-first (the Too Good To Go model)

- Out of scope for the PS. We mention it only to show we considered it.

4.2 Comparison

Direction	Real value	Tech depth	Differentiation	24h feasibility	Demonstrability	Measurable impact	Verdict
A Rule + broadcast	2	1	1	5	3	2	Baseline
B Batched assignment	3	3	2	4	3	3	Folded into E-stage-1
C Dynamic PDPTW	3	4	3	3	4	3	SHOULD (surge/insertion)
D Reliability layer	5	4	5	4	5	5	CORE
E Agentic LLM	2	2	2	3	3	1	Do not build
F RL	2	4	2	1	2	2	Do not build
G Forecasting	2	2	2	2	2	1	Replace with recurrence detection
H CV intake	2	3	2	3	4	1	Nice-to-have pre-fill only
I Conversational intake	4	2	4	4	5	3	MUST
J Marketplace	–	–	–	–	–	–	Out of scope

4.3 Decision

Relay = D (core) + I (intake) + A (baseline and fallback) + a simplified C (insertion into existing routes). Safety is handled by deterministic rules and fairness by a scoring term. The planner is a single pure function that both the live system and the simulator call.

Why not just C (routing) as the core? It is the "impressive" option, but it puts the effort into the wrong bottleneck. A perfect route for a volunteer who never accepts saves no food.

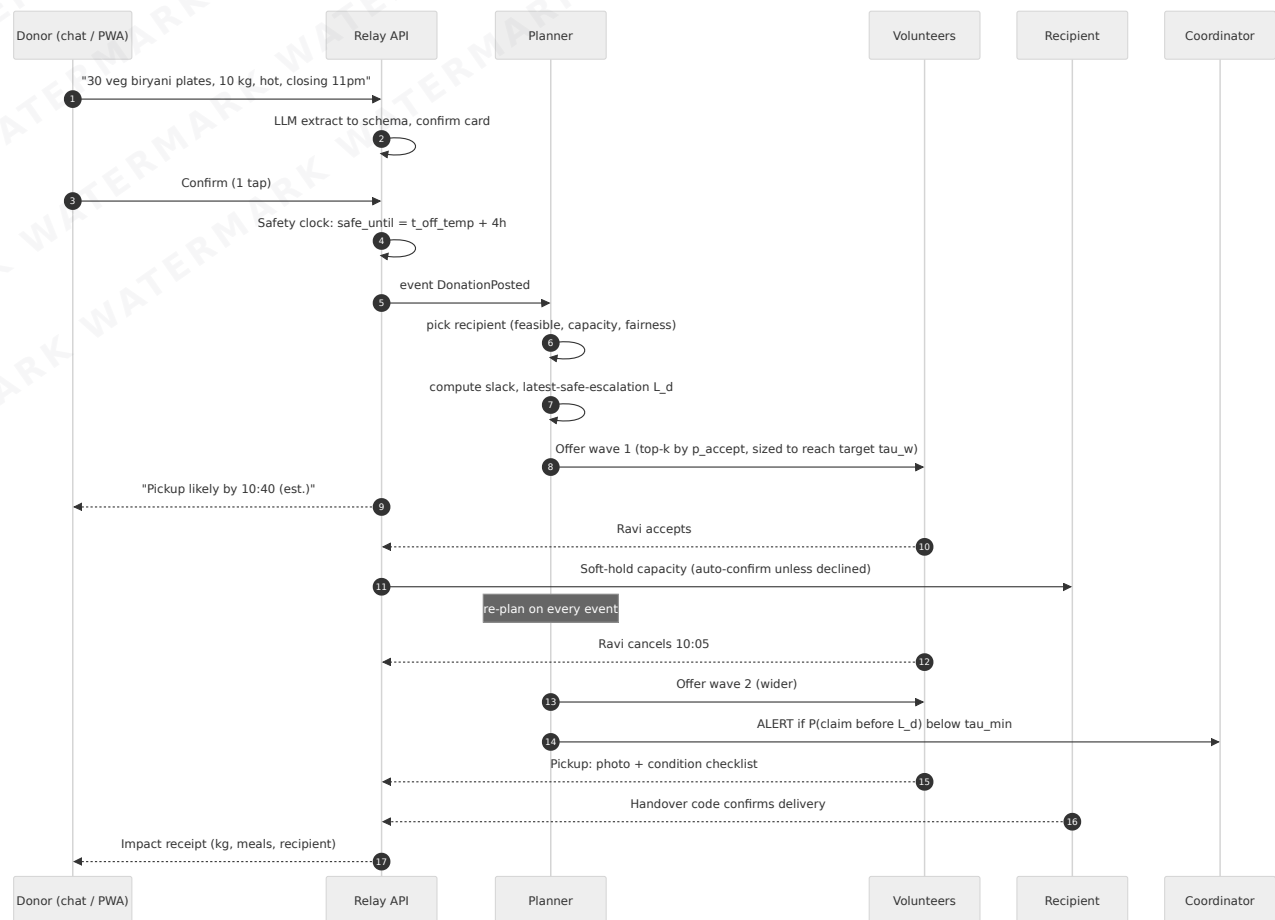
5. Final Solution: Relay

5.1 Product concept

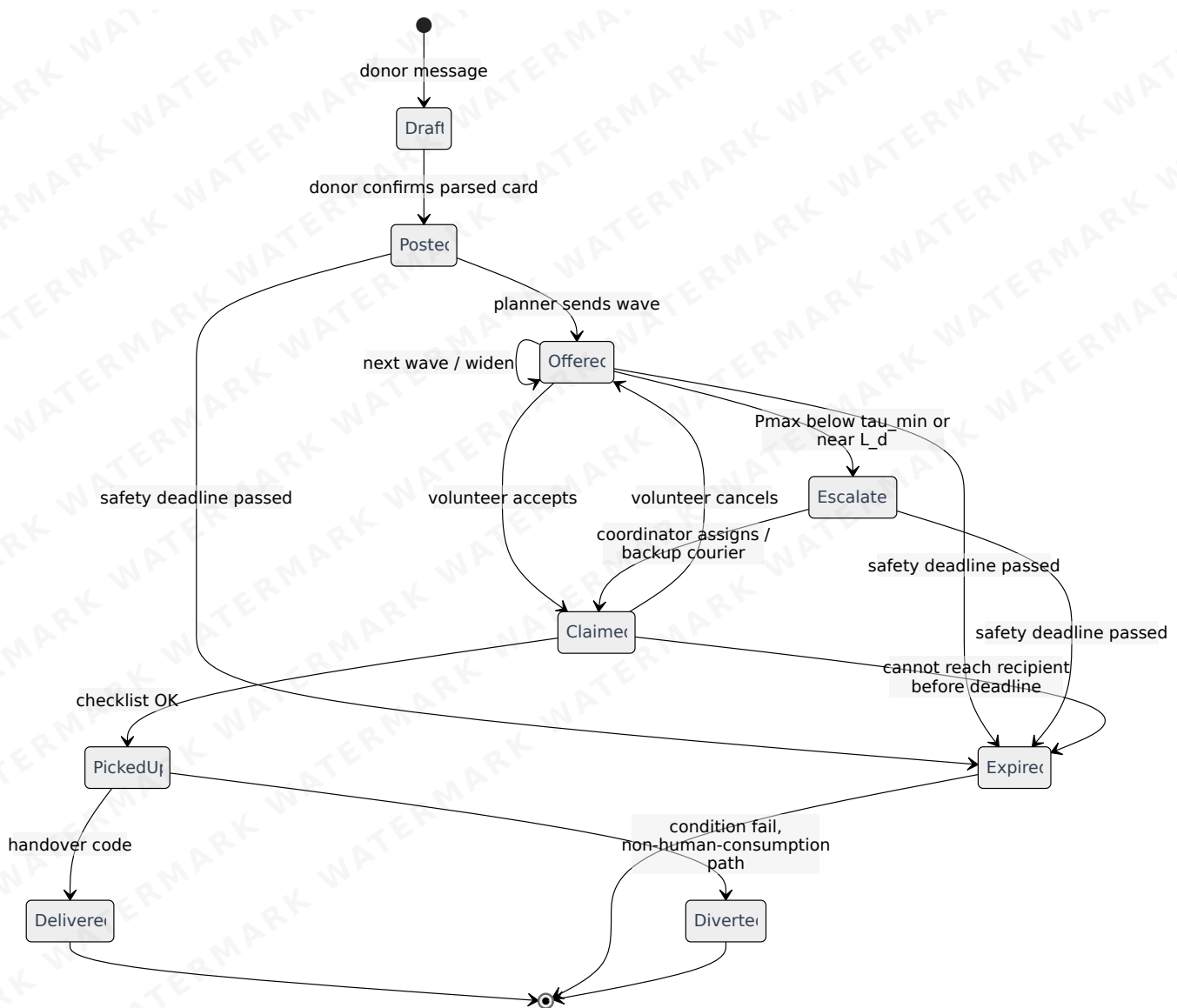
Relay is a dispatch co-pilot for food-rescue coordinators. For every donation it keeps a live answer to *"Will this food make it, and if not, when must a human step in?"* It automates the easy 80% and brings the risky 20% to the coordinator **early**.

[INF] **The primary user is the coordinator (dispatcher) at a food-rescue organisation or chapter.** Their pain is juggling WhatsApp threads and phone calls under deadline, and they are the one who adopts, configures and champions the tool. Donors, volunteers and recipients are *participants*. They should need **no app install**: chat and SMS, plus a lightweight PWA link.

5.2 Core workflow



5.3 Donation lifecycle (state machine)

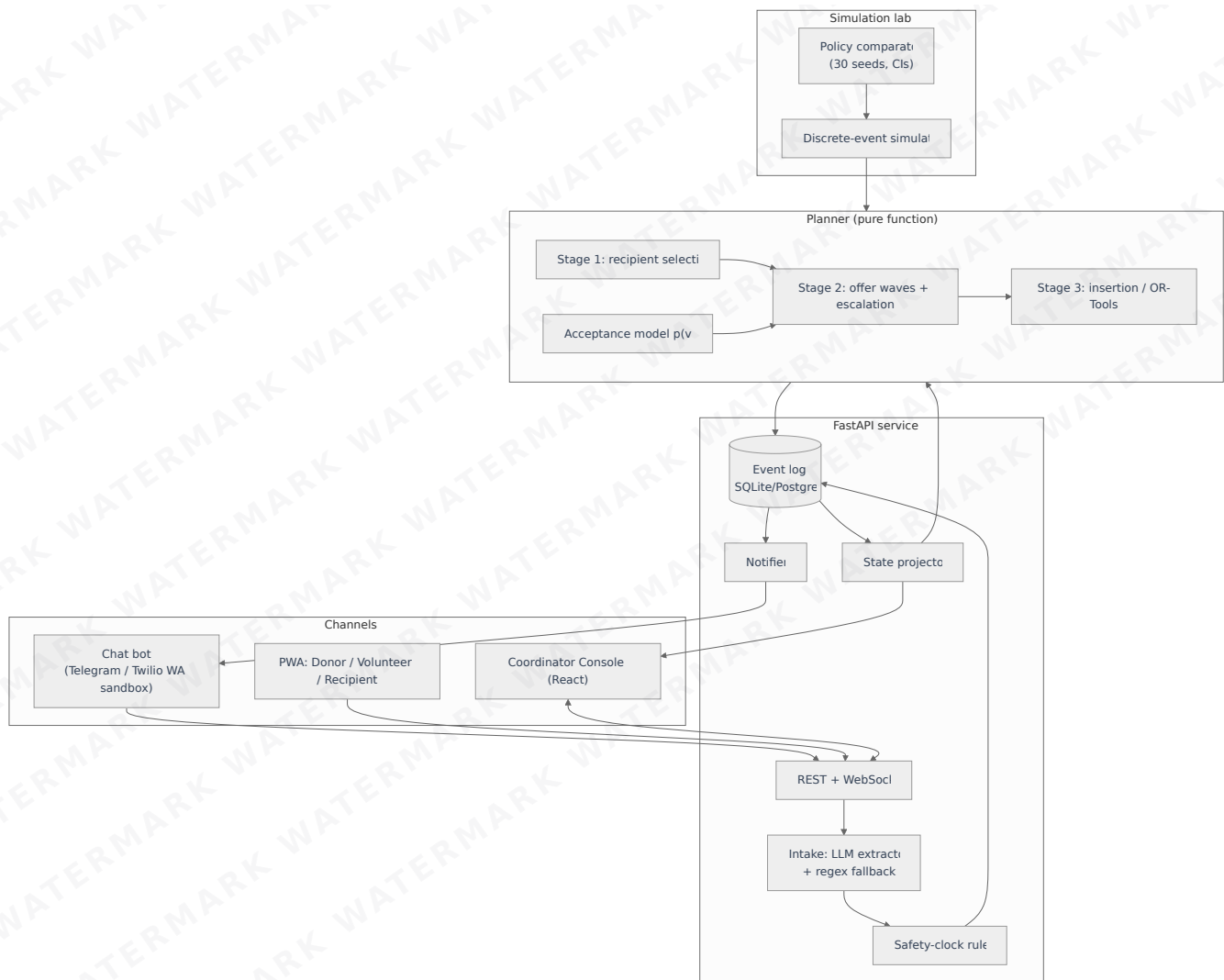


5.4 Components and why each exists

Component	Problem it solves	Why this approach	If removed
Chat/PWA intake + LLM extractor	Post in under 1 minute from messy text [PS]	Donors already type into WhatsApp. The LLM only extracts; a strict JSON schema and a confirm card catch errors, with a regex fallback	Donor friction goes up, contradicting the PS's ethics constraint on small donors
Safety-clock engine (rules)	"Never route expired-risk food" [PS]	Deterministic and auditable, based on FDA TPHC [R5]. Safety must never depend on a model	Safety depends on donor guesswork. Not acceptable
Recipient selector (stage 1)	Best-fit on capacity, need, distance and fairness [PS]	A small scored assignment problem. Fast and explainable	Nearest-only matching, which causes starvation and waste displacement
Acceptance model $p(v,d)$	Knowing who is likely to say yes	Logistic regression with per-volunteer Beta shrinkage. Calibrated and works with little data (§6)	Offers go out blind, so notifications are wasted or claims come late

Component	Problem it solves	Why this approach	If removed
Offer-wave policy + escalation (stage 2)	The primary bottleneck	Noisy-OR wave sizing against a per-donation deadline [R2], generalising 412FR's static scheme [R1]	Relay collapses into baseline A
Insertion / PDPTW re-optimiser (stage 3)	Surges and pooling onto volunteers already en route	Cheapest-insertion heuristic first; OR-Tools with a 1 s limit as an upgrade [R27]	Surges overflow; multi-stop (a PS "advanced" item) is missing
Event log (append-only)	Trust, audit, FSSAI/SB 1383 records [R6, R11], replay	Every state change is an event; state is a projection of the log	No audit trail and no deterministic simulation replay
Discrete-event simulator	We can't measure anything without real operations	It calls the <i>same</i> planner function as production, with common random numbers	Claims become unverifiable; the demo is just a UI
Coordinator console	Primary user's workflow	Risk queue sorted by slack, map, one-click escalation	The co-pilot has no cockpit
Impact ledger	Measurable diversion [PS]; ESG and CSR reports	Conservative method with stated factors	Impact numbers can't be defended

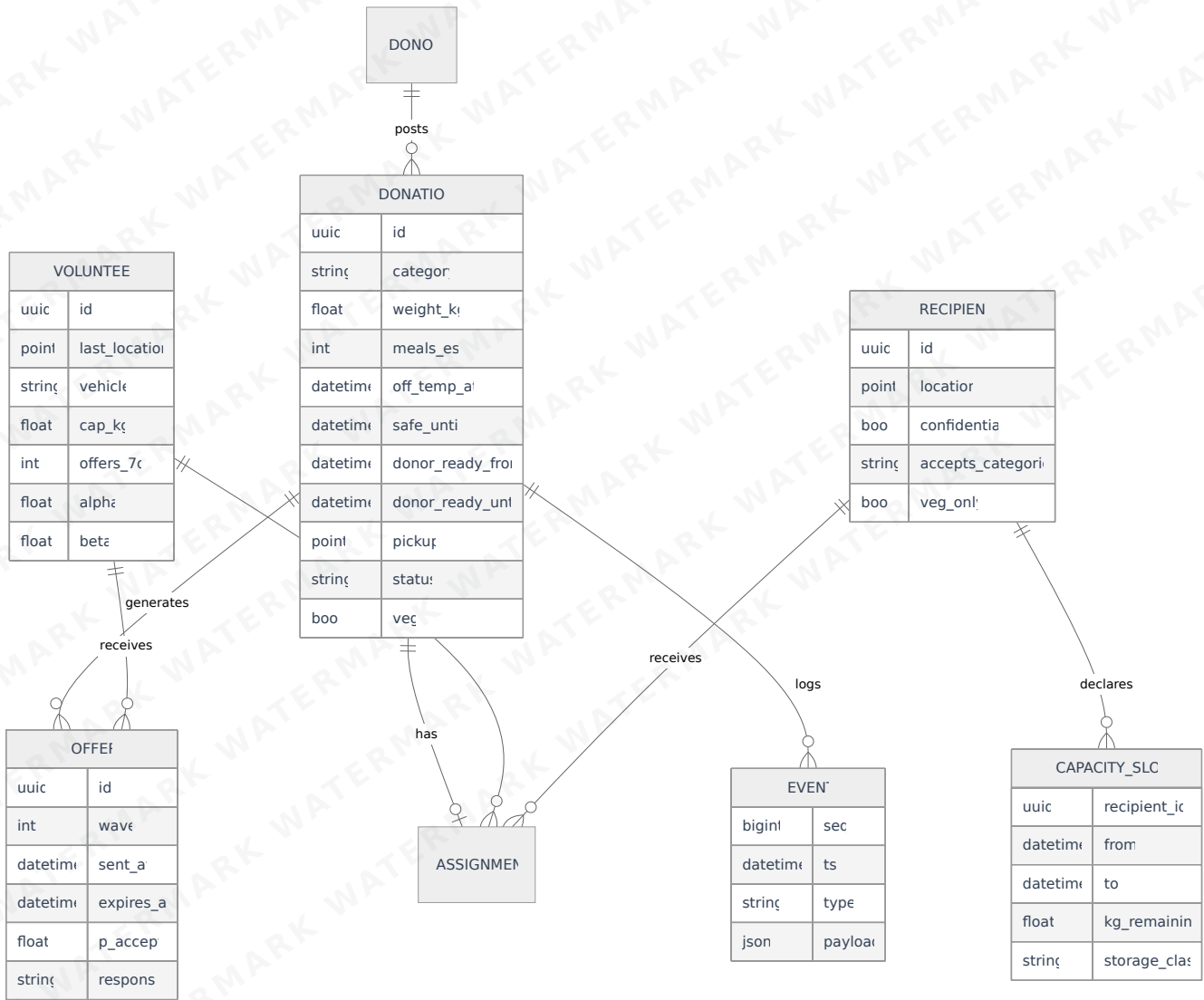
5.5 System architecture



Main design decision: the planner is a pure function `plan(state, now, config) -> Actions`. The live system calls it on every event and on a 60-second tick. The simulator calls it inside its event loop. That gives us:

1. one codebase to test;
2. a same-seed comparison of baseline and Relay that is fair by construction;
3. deterministic replays for debugging and the demo.

5.6 Data model (minimal)



6. AI / ML / RL: A Critical Evaluation

6.1 Principle

We use a model only when **(a)** a decision depends on something we can't observe directly, **(b)** a simpler rule would be measurably worse, and **(c)** a wrong prediction fails safely. Safety deadlines fail condition (c), so they are **rules**. Volunteer acceptance passes all three.

6.2 Component-by-component verdict

Candidate	Learn what?	Data needed	Verdict	If wrong
Volunteer acceptance $p(v,d)$	$P(\text{volunteer } v \text{ accepts donation } d \text{ within } \Delta)$	Offer logs (sent, responded, features). 412FR had years of this [R1]. We have only synthetic data	BUILD (small): logistic regression with per-volunteer Beta shrinkage	Waves are mis-sized → more notifications or later claims. Escalation still fires on time, because it depends on the deadline as well as p

Candidate	Learn what?	Data needed	Verdict	If wrong
Claim-risk classifier (as in [R1])	P(rescue ends unclaimed)	Same	Not built separately. It falls out of the noisy-OR over $p(v,d)$	–
LLM intake extraction	Text → {items, qty, unit, category, hot/cold, ready-until}	None to train; zero-shot with a schema	BUILD: extraction only, confirmed by the donor	The donor edits the card; regex fallback; the safety clock uses the <i>most conservative</i> category when unsure
Food-safety scoring via ML	"Is this safe?"	Labelled safety outcomes, which don't exist	DO NOT BUILD. Use rules [R5]	Health risk. Unacceptable
CV quantity/type	kg from a photo	Labelled photos	Nice-to-have pre-fill only	Capacity errors, so the donor must confirm
Supply forecasting	Donations by place and hour	Months of history (none, [R8])	Replace with recurrence detection (count-based)	–
Travel-time prediction	ETA	OSRM gives it	Use OSRM or haversine × detour factor	Mis-estimated slack, covered by a safety buffer
LLM explanations	"Why was shelter B chosen?"	–	SHOULD: templated text first, LLM optional	Cosmetic
RL for dispatch/notification	A policy mapping state → offers	A realistic environment and many episodes	DO NOT BUILD (§6.4)	–

6.3 The acceptance model

Prediction: $p_{v,d} = P(v \text{ accepts } d \text{ within } \Delta \mid \text{offered})$.

Model: a hierarchical logistic model. A shared feature effect is combined with a per-volunteer propensity, shrunk toward the population mean:

$$\text{logit } p_{v,d} = \theta_v + \beta_1 \text{dist}_{v,d} + \beta_2 \text{detour}_{v,d} + \beta_3 1[\text{late_night}] + \beta_4 \log(1 + n_v^{7d}) + \beta_5 \frac{w_d}{Q_v}$$

- θ_v is volunteer v 's baseline propensity. With little data, we estimate it by **empirical-Bayes shrinkage**: we keep a $\text{Beta}(\alpha_v, \beta_v)$ posterior over v 's raw acceptance rate, initialised from the population prior, and set $\theta_v = \text{logit} \left(\frac{\alpha_v}{\alpha_v + \beta_v} \right) - \overline{\text{logit}}$. New volunteers get the population mean. This handles **cold start**.
- n_v^{7d} is the number of offers in the last 7 days. It is the fatigue proxy, motivated by the burnout concern in [R1, R2].

- For the hackathon we fit it with scikit-learn `LogisticRegression` on simulator logs. **Honest note:** because the simulator's ground truth is itself a logistic model, the fit will look good by construction. **We present it as a pipeline demonstration, not as evidence of predictive power.**

What we evaluate: calibration comes before discrimination, because p values are *multiplied* inside the noisy-OR, so a miscalibrated p distorts every wave size. We report the Brier score, a reliability curve and AUC. The robustness ablation (§11) deliberately **misspecifies** p (noise, $\times 0.5$ and $\times 1.5$ bias) to show how the system degrades.

When it's wrong: the escalation deadline L_d (§9.4) depends only on travel times and the safety clock. **The worst case for a bad p is that the coordinator is alerted earlier or later than ideal. It is never later than L_d .** That is the safety net.

6.4 Is reinforcement learning justified? No, not for this build.

Question	Answer
Is the problem sequential with delayed effects?	Partly: notifications cause fatigue, and waves interact. In principle this suits RL.
Do we have a faithful environment?	No. Without real logs, the environment <i>is</i> our assumptions, so an RL agent would learn them back.
Is there evidence RL beats simple methods here?	In the food-rescue notification benchmark, DQN (1.55) < greedy (1.83) < Linear-Whittle (1.93) , normalised to random [R2]. The state of the art for this exact problem is structured bandits (RMAB, contextual budget bandits [R2, R3]), not deep RL.
Where does learning-plus-planning pay off?	At DiDi scale, with rich logs and millions of trips [R26]. Food rescue runs on the order of thousands of rescues a year [R1].
Explainability for coordinators?	Rolling-horizon optimisation with explicit slack is explainable. A neural policy isn't.
24h feasibility?	Training and tuning plus debugging a simulator-RL loop doesn't fit alongside the product.

Better alternatives, and what we use:

- **Now:** a greedy noisy-OR wave policy with deadline-driven escalation. Greedy is near-optimal here because the objective is monotone and we want the smallest set that reaches a target.
- **With 3–6 months of logs:** a **contextual bandit** (Thompson sampling on the logistic model) to explore volunteer targeting. Then **region-budgeted restless bandits** [R3] for equity-aware notification budgets.
- **Only if the org reaches ride-hailing-like volume:** learned value functions to guide dispatch [R26].

The line for judges: *"We looked at RL seriously. The published evidence in this exact domain has deep RL losing to greedy notification. We'd rather ship a transparent policy that escalates on time, and we've mapped the upgrade path to contextual bandits once real logs exist."*

7. Product Thinking

7.1 Users, pains and jobs-to-be-done

User	Biggest pain	Job-to-be-done	What makes them use Relay
Coordinator (primary)	Juggling threads; finding out too late that a rescue is failing; calling around at 10:45 pm	"Tell me which rescue needs me <i>now</i> , and let the rest run themselves"	A risk queue sorted by slack; automatic waves; one-click escalation; less phoning
Donor (restaurant/caterer)	"Will anyone actually come before we close?"; effort; liability worry	"Get rid of good food responsibly with zero hassle"	Posting by chat in 30 s; an honest ETA or an honest no ; impact receipt; audit trail
Volunteer	Irrelevant pings; wasted trips; unclear instructions	"Do good in a trip that fits my evening"	Fewer, better-targeted offers; clear route; pooled "+6 min, pick up more" offers
Recipient (shelter/kitchen)	Food arriving that they can't store, serve or that breaks dietary rules; surprise drops	"Receive what we can use, when we can receive it"	One-tap capacity toggles; category and veg-only rules; soft-holds; no surprise drops
ESG/CSR team; regulator	Numbers they can't audit	"Credible diversion reporting"	Event-sourced ledger; stated method; exportable records

7.2 Minimum useful product

One chapter, one city, and a coordinator who runs every evening's rescues through Relay: chat intake, safety clock, recipient selection, offer waves, escalation alerts, proof of delivery, and a weekly impact export. **Routing optimisation is not part of the minimum useful product. The reliability layer is.**

7.3 Where Relay differs from the alternatives

	WhatsApp group	412FR scheme (2018–19, as published) [R1]	Food Rescue US [R13]	Copia [R4]	Relay
Intake	Free text	App form	Web app	App	Chat free text → structured, confirmed
Claim mechanism	Whoever replies	Radius → all after 15 min → dispatcher in last hour (static)	Self-schedule / adopt recurring	Managed logistics (paid)	Per-donation waves sized to deadline and p(accept)
Knows when a rescue is failing	No	By fixed rule	Human	Internal	Live P(claim before L_d) and slack
Recipient capacity	Ask around	Platform-managed	Agency needs posted	Nonprofit network	Time-slotted capacity + soft-holds
Re-plan on disruption	Manual	Manual	Manual	Internal	Event-triggered

	WhatsApp group	412FR scheme (2018–19, as published) [R1]	Food Rescue US [R13]	Copia [R4]	Relay
Audit / records	None	Platform	Reports	Tax/ESG dashboards	Event-sourced ledger

Caveat [INF]: proprietary platforms may have changed since the published descriptions. Food Rescue Hero has since worked with researchers on recommenders and bandits [R2, R3]. So we claim **differentiation from the publicly documented workflows**, not uniqueness.

7.4 What could become a real product versus what is a hackathon gimmick

- **Real:** the reliability and escalation engine; chat-native intake; recipient capacity holds; the audit ledger; converting recurring donors into scheduled rescues; CSR/ESG-grade reporting.
- **Gimmick (avoid):** "AI freshness detection" from photos; an LLM negotiating pickups; blockchain provenance; gamified leaderboards as the headline feature; a hand-waved "AI predicts hunger hotspots."

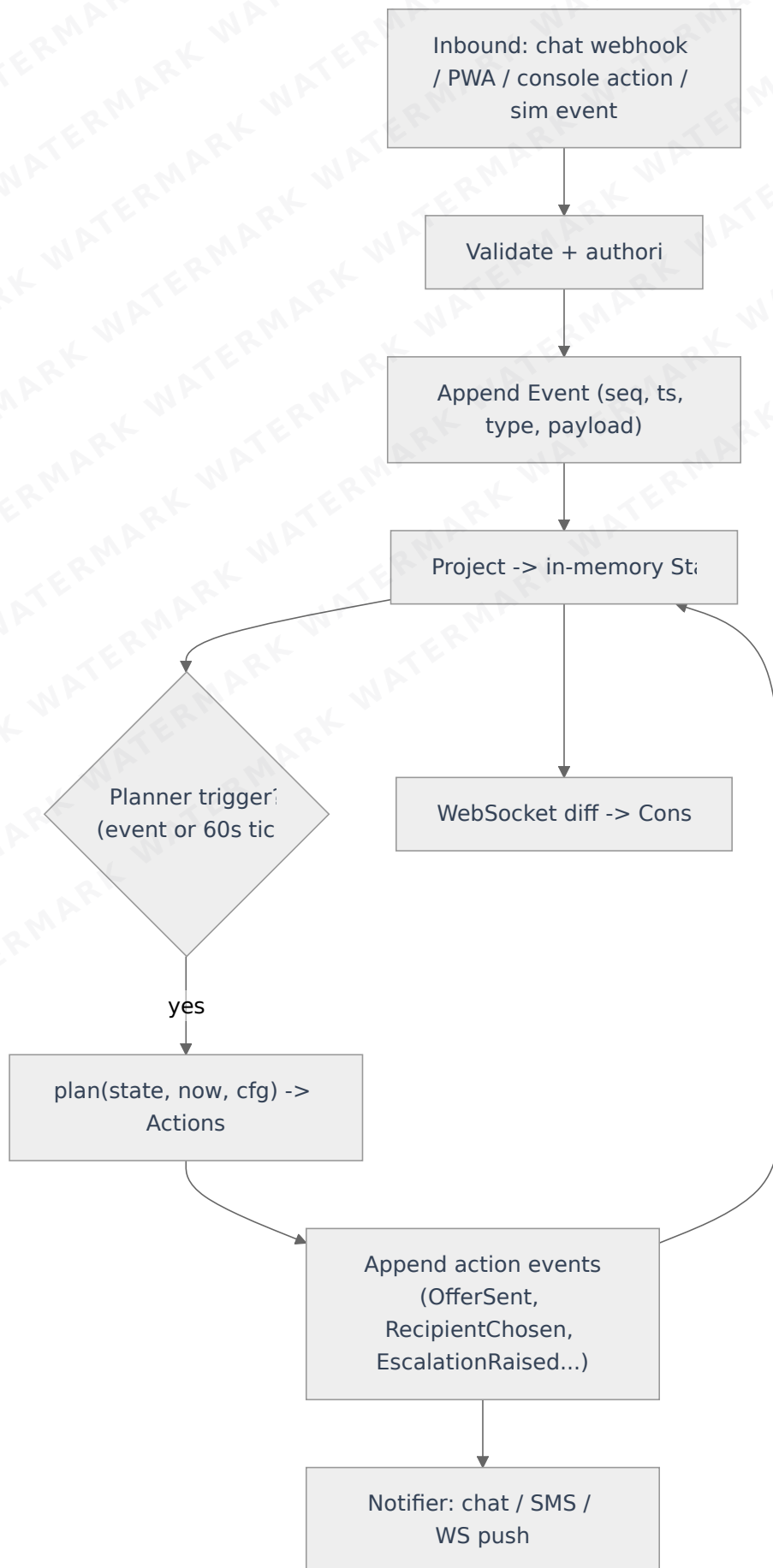
8. Technical Design

8.1 Stack, chosen for 24 hours

Layer	Choice	Why	Rejected alternative
Backend	Python 3.11 + FastAPI	OR-Tools, scikit-learn and the simulator are all Python, so one language serves both planner and API	Node: splits the planner and sim across languages
DB	SQLite (WAL mode) for the hackathon; Postgres in production	No setup, and the event log is append-only	PostGIS: not needed; haversine over roughly 100 points is trivial
Realtime	FastAPI WebSocket , broadcasting projected state diffs	Built in	Kafka/Redis: unnecessary at demo scale
Planner	Pure Python + OR-Tools (optional stage)	Anytime behaviour, with greedy as the fallback	Gurobi: licence friction
ML	scikit-learn LogisticRegression + NumPy Beta updates	Small and calibrated	XGBoost/NN: no data to justify them
LLM	Any hosted LLM with JSON-schema / structured output	Extraction only	Fine-tuning: pointless without data
Frontend	React + Vite + Leaflet (OSM or CARTO tiles), Tailwind	Fast to build maps	Mapbox: needs keys and billing
Chat channel	Telegram Bot (free, instant) <i>or</i> Twilio WhatsApp Sandbox (no Meta approval)	The demo must reach phones	WhatsApp Cloud API production: approval

Layer	Choice	Why	Rejected alternative
	needed; users must <code>join</code> first; 1 msg/3 s; sessions expire after 3 days) [R31]		takes too long
Maps/ETA	OSRM table, precomputed and cached (demo server ≤ 1 req/s, non-commercial [R29]); fallback haversine $\times 1.35$	Real road times without hammering the server	Google Distance Matrix: billing
Deploy	Single VM or Render/Railway for the API, Vercel/Netlify for the frontend; local fallback on a laptop	The demo must not depend on venue Wi-Fi	Kubernetes: absurd for 24h

8.2 Services and data flow



Event types: DonationDrafted, DonationPosted, SafetyClockSet, RecipientChosen, CapacityHeld, CapacityReleased, OfferSent, OfferAccepted, OfferDeclined, OfferExpired, ClaimCancelled, EscalationRaised, EscalationResolved, PickedUp, ConditionFailed, Delivered, Expired, Diverted, RecipientCapacityUpdated, VolunteerLocationUpdated, Tick.

8.3 API surface (MVP)

Method	Path	Purpose
POST	/intake/parse	Free text → draft donation (LLM + fallback)
POST	/donations	Confirm or post a donation (form or confirmed draft)
GET	/donations/{id}	Status, safety clock, plan, P(claim), L_d
POST	/offers/{id}/respond	accept / decline (from chat button or PWA)
POST	/assignments/{id}/cancel	Volunteer cancels
POST	/assignments/{id}/pickup	Checklist (temp OK, packaging OK, photo)
POST	/assignments/{id}/deliver	Handover code from the recipient
PATCH	/recipients/{id}/capacity	Toggle or adjust capacity slots
POST	/escalations/{id}/resolve	Coordinator assigns a person or backup
GET	/impact?from&to	Ledger aggregates + CSV export
WS	/ws/ops	Live state diffs for the console
POST	/sim/run	Run a scenario × policy × seeds
GET	/sim/compare	Paired metrics + CIs
POST	/webhook/telegram OR /webhook/twilio	Chat inbound

8.4 Security and privacy

- **Location minimisation:** an offer shows the pickup *area* (a roughly 500 m cell) and the distance. The exact address is revealed only after acceptance. The volunteer's live location is stored only while an assignment is active.
- **Confidential recipients** (for example domestic-violence shelters): flag `confidential=true`. Their address is never shown to volunteers; handover happens at a designated proxy point, or staff collect.
- **Auth:** magic-link or OTP per role. Chat users are identified by their chat ID once linked. Handover codes are 4-digit, single-use and expire after 2 hours.
- **LLM data:** send only the donation text, stripped of phone numbers and emails by regex before the call.
- **Rate limits** on public webhooks. Secrets live in environment variables.

8.5 Reliability and failure handling (engineering)

Failure	Handling
OR-Tools slow or infeasible	Hard <code>time_limit</code> of 1 s; the greedy plan is always computed first and used if OR-Tools returns nothing better
LLM down or slow (>3 s)	Regex/keyword parser + form fallback; the donor never waits on the LLM

Failure	Handling
OSRM unavailable	Cached matrix → haversine × detour factor, with the ETA buffer widened by 20%
Chat provider down	SMS fallback; the console shows an "undelivered offers" banner
Planner exception	Catch it, log it, keep the last valid plan, raise a coordinator alert (fail loud)
Server restart	State is rebuilt by replaying the event log (deterministic)

8.6 Monitoring (production path; minimal in the hackathon)

Planner latency p50/p95, offers per donation, time-to-claim, escalations and their *lead time before deadline*, expiries, notifications per volunteer per week, per-zone rescue rate. In the hackathon these are the **same metrics the simulator reports**, shown in the console.

8.7 Scaling beyond one city (PS constraint)

The problem is geographically separable: rescues rarely cross a city. So we **partition by city or zone**. Each partition has its own planner worker, and the event log is keyed by partition. The planner is stateless, since state is a projection, so it scales horizontally. At roughly 10^3 open donations per zone, stage 1 is still milliseconds; OR-Tools gets windowed to the most urgent N.

9. Mathematical Formulation

9.1 Notation

Symbol	Meaning
$\mathcal{D}_t$	Open donations at time t
w_d, m_d	Weight (kg); meals. m_d = plate count if given, otherwise $w_d/0.544$ (the 1.2 lb/meal convention used by Feeding America and ReFED [R9, R32]; configurable)
k_d, g_d	Food category; storage class (hot / cold / ambient)
$o_d; [a_d, b_d]$	Pickup location; donor availability window
t_d^{off}	Time the food left temperature control (conservative default: the posting time)
$H(k_d)$	Safe holding horizon: 4 h (TPHC), 6 h (cold ≤ 21 °C, never exceeded), longer for ambient-stable categories [R5, ASM]
$s_d = t_d^{\text{off}} + H(k_d)$	Safety deadline (hard)
$\mathcal{R}; q_r; [\alpha_r, \beta_r]$	Recipients; location; receiving window
$c_{r,g}$	Remaining capacity (kg) at r for storage class g
μ_r	Recipient's <i>use lead</i> time from receipt to service (0 if serving on arrival)

Symbol	Meaning
F_r	Fill rate = meals received ÷ declared need over a rolling 7 days
$\mathcal{V}_t; \ell_v; Q_v$	Available volunteers; location; vehicle capacity
$p_{v,d}$	P(v accepts d within the wave timeout Δ) (§6.3)
n_v	Offers to v in the last 7 days; budget K
$\tau(x, y); \sigma$	Travel time; service (handling) time
ε	Safety buffer (for example 20 min)

9.2 Feasibility of a (volunteer, donation, recipient) triple

A triple (v, d, r) is feasible at time t if all of the following hold:

$$\begin{aligned}
t^{\text{pick}} &= \max(t + \rho_v + \tau(\ell_v, o_d), a_d) \leq b_d \\
t^{\text{drop}} &= \max(t^{\text{pick}} + \sigma + \tau(o_d, q_r), \alpha_r) \leq \beta_r \\
t^{\text{drop}} + \mu_r &\leq s_d - \varepsilon \quad (\text{food is used before the safety deadline}) \\
k_d &\in A_r, \text{ veg/dietary rules satisfied, } w_d \leq \min(Q_v, c_{r,g_d})
\end{aligned}$$

Here ρ_v is v's expected response delay. The third line is the subtle one. **The safety deadline applies to when the food is used, not when it is delivered.** A shelter whose next meal is at 7 am can't accept 4-hour food at 11 pm unless the cold chain was kept, which is a configurable category rule. [INF] Naive systems miss this.

Define the **slack** of donation d with recipient r as the latest pickup time that still meets every constraint, minus now. It is the single most important number on the coordinator's screen.

9.3 Stage 1: recipient selection (small assignment problem)

Decision $x_{dr} \in \{0, 1\}$:

$$\begin{aligned}
&\max_x \sum_{d \in \mathcal{D}_t} \sum_{r \in \mathcal{R}} x_{dr} \left[m_d (1 + \lambda_F (1 - F_r)) - \lambda_T \tau(o_d, q_r) - \lambda_S (S_{\min} - \widehat{\text{slack}}_{dr})^+ \right] \\
&\text{s.t. } \sum_r x_{dr} \leq 1 \quad \forall d, \quad \sum_{d: g_d=g} w_d x_{dr} \leq c_{r,g} \quad \forall r, g, \quad x_{dr} = 0 \text{ if infeasible for every } v.
\end{aligned}$$

- $\widehat{\text{slack}}_{dr}$ uses a **robust pickup estimate**: the ETA of the 3rd-nearest eligible volunteer, not the nearest, which avoids optimism.
- The $(1 - F_r)$ term implements the equity idea of maximising the minimum fill rate from Lien et al. [R23] as a soft priority.
- This is a generalised assignment problem, NP-hard in general, but instances are tiny (tens × tens). **Solver**: greedy in increasing order of slack, most urgent first, taking each donation's best feasible recipient. Optional exact solve with OR-Tools CP-SAT.
- Hysteresis**: once a recipient has been communicated for d , switch only if the current one becomes infeasible or the score improves by more than δ . Humans hate churn.

9.4 Stage 2: deadline-aware offer waves and escalation (the core)

Latest safe escalation time. Let B be the org's backup resource: a coordinator's on-call regular, an org vehicle, or a paid courier, with response time ρ_B and location ℓ_B . Then

$$L_d = \min \left(b_d, \min(\beta_r, s_d - \varepsilon - \mu_r) - \sigma - \tau(o_d, q_r) \right) - \rho_B - \tau(\ell_B, o_d) - \varepsilon_{\text{esc}}.$$

L_d is **the last moment at which handing the rescue to the backup still gets the food used safely**. It depends on travel times and rules only, not on the ML model.

Claim probability for a wave set. Assume responses are independent, the same assumption as [R2]:

$$P_d(S) = 1 - \prod_{v \in S} (1 - p_{v,d}).$$

Per-wave target from remaining time. With wave timeout Δ , the number of waves that fit before escalation is $W_d = \lfloor (L_d - t) / \Delta \rfloor$. To reach an overall target τ^* (for example 0.95) by L_d , treating waves as independent, each wave needs

$$\tau_w = 1 - (1 - \tau^*)^{1/W_d}.$$

This is the mechanism in one line. Lots of slack means many waves, so a small τ_w and few notifications per wave. Little slack means a large τ_w and a wider blast. No slack means escalate now. By contrast, a static scheme spends the same notifications on a bakery donation with 20 hours of slack as on biryani with 90 minutes.

Wave selection. Choose the smallest $S \subseteq E_d$ with $P_d(S \cup O_d) \geq \tau_w$, where E_d is the eligible, not-yet-offered, under-budget volunteers and O_d is the outstanding offers:

```
sort E_d by p_{v,d} / (1 + \gamma \cdot n_v) descending # discount heavily-notified volunteers
S = {}
while P_d(S \cup O_d) < \tau_w and S.size < k_max: S.add(next)
```

Because P_d is monotone in S and the aim is the smallest set that reaches the threshold, sorting by p is optimal when $\gamma = 0$. With the fatigue discount it is a sensible heuristic.

Early escalation rule. Escalate at time t if

$$t \geq L_d - \Delta_{\text{alert}} \quad \text{or} \quad P_d^{\max} = 1 - \prod_{v \in E_d \cup O_d} (1 - p_{v,d}) < \tau_{\min}^*.$$

The second condition is what makes Relay *early*. For example, in a suburb at 11 pm with two eligible volunteers, even notifying everyone gives $P^{\max} \approx 0.5$. **The coordinator hears about it the minute it is posted, not in the last hour.** This is the mechanism by which we expect a benefit in low-supply zones, the equity problem in [R3]. [INF; to be measured in §11]

Independence caveat. Correlated behaviour (rain, a festival night, everyone asleep) breaks independence and makes P_d optimistic. Mitigations: a conservative multiplier on p at night or in bad weather, and a deadline-based L_d that fires regardless.

9.5 Stage 3: pooling and multi-stop (insertion, then optional PDPTW)

Cheapest feasible insertion for a volunteer v with current route R_v (a sequence of stops with deadlines): insert the pair $(o_d \rightarrow q_r)$ at positions $i \leq j$ to minimise added time, subject to every existing stop still meeting its §9.2 constraints and the load staying within capacity. The cost is $\mathcal{O}(|R_v|^2)$ per volunteer, and routes have 2–6 stops. If the added time is at most θ_{detour} (for example 10 min), it becomes an **offer** to v ("7 min: grab 8 kg rotis from X on your way"), and it enters the wave set with $p'_{v,d}$ via the detour feature. **Accepted work is never reassigned without consent.**

Full re-optimisation (OR-Tools, optional). For surge "what-if" planning we solve a PDPTW with prize collection:

$$\min \sum_v \sum_{(i,j)} \tau_{ij} y_{ijv} + \sum_{d \in \mathcal{D}^{\text{new}}} \pi_d (1 - z_d) \quad \text{with} \quad \pi_d = M \cdot m_d$$

subject to the standard routing constraints: flow conservation per vehicle; pickup and delivery on the same vehicle with pickup first; time windows $T_i \in [e_i, l_i]$ derived from §9.2 as **hard** windows; load within Q_v ; committed pairs pinned to their volunteer ($z_d = 1$, allowed-vehicle restriction); new pairs optional through **disjunctions with penalty** π_d [R27].
 Solver: `PARALLEL_CHEAPEST_INSERTION` for the first solution, then `GUIDED_LOCAL_SEARCH` with a 1 s time limit, warm-started from the greedy plan. The output is a *set of proposed insertion offers*, not orders.

9.6 Complexity and latency budget

Stage	Complexity	Target latency (50 donations, 60 volunteers, 12 recipients)
Feasibility matrix	$\mathcal{O}(D V R)$ with cached τ	< 20 ms
Stage 1 greedy	$\mathcal{O}(D R + D \log D)$	< 5 ms
Stage 2 waves	$\mathcal{O}(D V \log V)$	< 10 ms
Stage 3 insertion	$\mathcal{O}(D V \bar{L}^2)$	< 20 ms
OR-Tools (optional)	bounded by time limit	≤ 1 s

These are design targets; measure them (§11).

9.7 Scalar objective, used only for tuning

For tuning λ and τ^* in simulation:

$$J = \underbrace{\sum_d m_d 1[\text{used safely}]}_{\text{meals}} - \lambda_N \# \text{notifications} - \lambda_E \# \text{escalations} - \lambda_C \# \text{courier uses} - \lambda_G \left(\max_z \text{miss}_z \right)$$

Safety violations are **not** in J . They are constrained to zero. The z -terms are per-zone miss rates, used as an equity penalty.

10. Data & Simulation

10.1 Data inventory

Data	Type	Source	Use
Donor and recipient locations	Real (public)	OpenStreetMap via Overpass (amenity=restaurant/fast_food/cafe, shop=supermarket/bakery, amenity=social_facility, shelters, community kitchens), for one chosen city	Realistic spatial layout
Road travel times	Real (public)	OSRM table (cached) [R29]; fallback haversine $\times 1.35$	ETAs, feasibility
Safety horizons	Real (regulatory) + [ASM]	FDA TPHC 4 h / 6 h [R5]; ambient/produce horizons assumed	Safety clock
Meal conversion	Real (convention)	1.2 lb = 1 meal [R9, R32]	Impact
Difficulty anchor	Real (published)	412FR: 16.4% of rescues unclaimed or last-hour [R1]	Calibrate the baseline
Donation arrivals, sizes, categories	Synthetic [ASM]	Parameterised generator	Load
Volunteer availability and acceptance	Synthetic [ASM]	Ground-truth logistic + heterogeneity	Behaviour
Recipient hours, capacities, needs	Synthetic [ASM]	Parameterised	Constraints
CO2e factor	Configurable	Pick and cite one: ReFED factors (Quantis-based, per food type) [R9] or EPA tools [R10]	Impact (labelled as an estimate)

10.2 Simulator design (discrete-event)

Engine: a `heapq` priority queue of timestamped events, or SimPy. Single-threaded and deterministic given a seed.

Processes:

1. **Donation generator:** a non-homogeneous Poisson process with rate $\lambda(t)$, with peaks around post-lunch and close of business (restaurants about 22:00–23:30). Donor type is sampled from the mix below. [ASM]
2. **Volunteer agents:** shift availability, home location (residential OSM zones), and ground-truth acceptance $\text{logit } p_{v,d}^{\text{true}} = \theta_v^{\text{true}} - 0.35 \text{dist}_{v,d}[\text{km}] - 0.8 \mathbb{1}[\text{late}] - 0.3 \log(1 + n_v) + \epsilon$ with $\theta_v^{\text{true}} \sim \mathcal{N}(0.2, 0.8^2)$ [ASM]; response delay $\sim \text{LogNormal}(\text{median } 4 \text{ min})$; $P(\text{cancel after accept}) = 0.08$; $P(\text{no-show}) = 0.03$ [ASM].
3. **Recipients:** windows (some 24 h, most about 09:00–21:00), per-class capacity, veg-only flag, use lead μ_r , stochastic "we're full" events.
4. **Backup:** coordinator/courier with response $\rho_B = 20 \text{ min}$, always available, costly (counted).
5. **Travel:** OSRM or haversine; the time-of-day speed factor slows peak hours [ASM].
6. **Disruption injector:** scripted events (cancellations, capacity drops, surge posts, a "rain" multiplier on p).

Donor mix [ASM]:

Type	Share	Category	kg	Horizon
Restaurant	45%	cooked, hot	3–15	4 h
Caterer / event	10%	cooked, hot/cold	10–40	4–6 h
Campus dining	15%	cooked	10–25	4 h
Bakery	15%	ambient	2–8	24 h
Grocer	15%	produce/package	5–30	48 h+

Calibration step (important for honesty): tune the arrival rate and volunteer density so that **baseline B1** (the 412-style static scheme) produces roughly **15–17% "negative" rescues**, matching the published 16.4% [R1]. This does *not* validate Relay. It makes sure the scenario is not rigged to be easy or impossibly hard.

Scenarios: `normal_weekday`, `friday_night` (the demo), `surge` (3× donations), `volunteer_drought` (50% fewer), `recipient_closure` (the two largest recipients go offline at 21:00), `rain` ($p \times 0.6$), `sparse_suburb` (a periphery zone with thin volunteer supply).

Common random numbers: each (scenario, seed) pre-generates *all* exogenous randomness: arrivals, volunteer shifts, the uniform draw that decides each potential accept, cancel draws. Every policy then faces an identical world. This removes most variance from paired comparisons, and it is also what makes the side-by-side demo fair.

11. Evaluation Framework

11.1 What success means

In simulation, Relay should rescue **more safe, usable food** with **fewer or equal volunteer notifications**, escalate **earlier** with **fewer late interventions**, and have **zero safety violations**. It should do this without making zone inequity worse, and keep planner latency under one second.

11.2 Metrics

Metric	Definition	Why it matters
Rescue rate (kg)	kg used safely ÷ kg posted	The primary outcome
Meals rescued	$\sum m_d$ used safely	The headline number for the pitch
Expired kg	kg that reached s_d unrescued	The loss we're fighting
Safety violations	Deliveries where $t^{\text{drop}} + \mu_r > s_d$	Must be 0 (hard constraint check)
Notifications / rescue	Offers sent ÷ rescues completed	Volunteer burden [R1, R2]
Max offers / volunteer / week	95th percentile of n_v	Burnout tail
Time to claim	Median and p90 of claim time – post time	Donor experience

Metric	Definition	Why it matters
Escalations	Count and lead time (L_d – escalation time)	Earlier is better; late escalations are failures
Backup/courier uses	Count	Cost proxy
Recipient fairness	Min fill rate; Gini of meals ÷ need	Equity [R23]
Zone gap	max – min rescue rate across zones	Geographic equity [R3]
Plan churn	Recipient changes after the donor was notified	Human trust
Planner latency	p50/p95 ms	"Near-instant" [PS]
Acceptance model	Brier, calibration curve, AUC	Quality of p

11.3 Baselines (all use the same safety rules, simulator and seeds)

ID	Policy	Represents
B0 Broadcast	Notify all volunteers at once; first accept wins; nearest feasible recipient; no re-planning; coordinator acts only when the food is 30 min from expiry	A WhatsApp group
B1 Static tiered	Notify within 8 km (≈ 5 mi) → notify all after 15 min → coordinator alert when 60 min of the pickup window remain (the published 412FR 2018–19 defaults [R1])	The best documented real practice
B2 Optimiser-only	Stage 1 + stage 3 routing, with offers sent to the <i>optimal</i> volunteer one at a time and no probability model	"Routing-first" hackathon approach
Relay	Full stages 1–3 + p model + deadline-aware waves + early escalation	Ours

11.4 Hypotheses (pre-registered, written before running)

- **H1:** Relay's rescue rate is at least B1's, with $\geq 20\%$ fewer notifications per rescue. (*Tests whether deadline-aware sizing cuts spam without cutting rescues.*)
- **H2:** Relay has fewer *late* escalations (lead time < 10 min) than B1.
- **H3:** In `sparse_suburb`, Relay narrows the zone gap compared with B1.
- **H4:** Under `recipient_closure` and `surge`, Relay's expired kg is lower than B0/B1's.
- **H5:** Removing the p model (uniform p) loses most of Relay's notification savings; removing escalation loses most of its rescue gains.

If a hypothesis fails, **report it**. A failed H1 with an honest explanation is stronger with serious judges than numbers that look too good.

11.5 Ablations

Variant	Removes	Question
Relay – p model	Uses a constant p = population mean	Does learning <i>who</i> is likely to accept matter?
Relay – early escalation	Escalates only at $L_d - \Delta_{\text{alert}}$	Value of the P^{max} rule
Relay – deadline sizing	A fixed τ_w	Value of $\tau_w(W_d)$
Relay – fatigue discount	$\gamma = 0$, no budget K	Burnout versus rescue trade-off
Relay – insertion	No pooling offers	Value of multi-stop
Relay + OR-Tools	Full PDPTW re-optimisation	Does exact routing beat insertion at this scale? (<i>We expect little difference; report whatever the result is.</i>)
Relay – μ_r (use-lead)	Deadline on delivery only	How many "delivered but unusable" rescues the naive model produces

11.6 Robustness and stress tests

- **p misspecification:** multiply the predicted p by 0.5 or 1.5, or add $N(0, 0.5)$ logit noise. Expect graceful degradation, with L_d still bounding the damage.
- **Correlated behaviour:** in the *rain* scenario the true p falls, but the model doesn't know.
- **Missing data:** 20% of donations have no category (the default is the shortest horizon); recipient capacity is stale (not updated in 4 h).
- **Multiple simultaneous events:** cancellation + closure + surge within 10 minutes (the demo scenario).
- **Scale:** 10× donors and volunteers, to measure planner latency.
- **Adversarial donor:** reposts the same food, or posts an absurd quantity (1,000 kg) → validation, plus coordinator review above a threshold.

11.7 Statistical protocol

- 30 seeds per (scenario, policy), with common random numbers.
- Report the mean $\pm$ 95% CI of the **paired difference** (Relay – baseline) per metric, using a bootstrap over seeds.
- Include one **distribution plot**, not just means, for expired kg per night.
- Put the label "*Simulated; parameters in Appendix; calibrated to 1 published statistic*" on every results slide.

11.8 Things we must not claim

- "Relay reduces food waste by X% in the real world."
- "Our model predicts volunteer behaviour with Y% accuracy" (the data is synthetic).
- A CO2e figure without its method and factor stated next to it.

12. 24-Hour Hackathon Strategy

12.1 Scope tiers

MUST BUILD (without these there is no project)

1. Event log + state projection (SQLite) and the donation lifecycle state machine
2. Safety-clock rules table and feasibility checks (§9.2, including μ_r)
3. Planner v1: stage 1 greedy + stage 2 waves/escalation + a constant-p fallback
4. Simulator with the `friday_night` scenario, the B0/B1 baselines and common random numbers
5. Coordinator console: map, donation cards with a **countdown to the safety clock**, P(claim) gauge, risk queue sorted by slack, escalation alerts
6. Donor intake: web form (4 fields + smart defaults) **and** free-text paste → LLM extraction → confirm card
7. Volunteer and recipient PWA views: offer accept/decline, pickup checklist, delivery code, capacity toggles
8. Impact panel (kg, meals, with the method shown)
9. The side-by-side same-seed comparison view (the core of the demo)

SHOULD BUILD

- Acceptance model (logistic + Beta shrinkage) trained on simulator logs, with a calibration plot
- Stage 3 insertion/pooling offers
- Real chat channel (Telegram bot, or the Twilio WA sandbox) for the donor post and volunteer offer on a real phone
- 30-seed batch comparison with CIs and a results table in the UI
- Recurring-donor detection → "propose a scheduled rescue" card
- OSM-based city layout + a cached OSRM matrix

NICE TO HAVE

- OR-Tools PDPTW re-optimisation (surge "what-if")
- Photo pre-fill of category (vision LLM), with a confirmation step
- Hindi UI strings; SMS fallback
- CSV/PDF impact receipt for donors

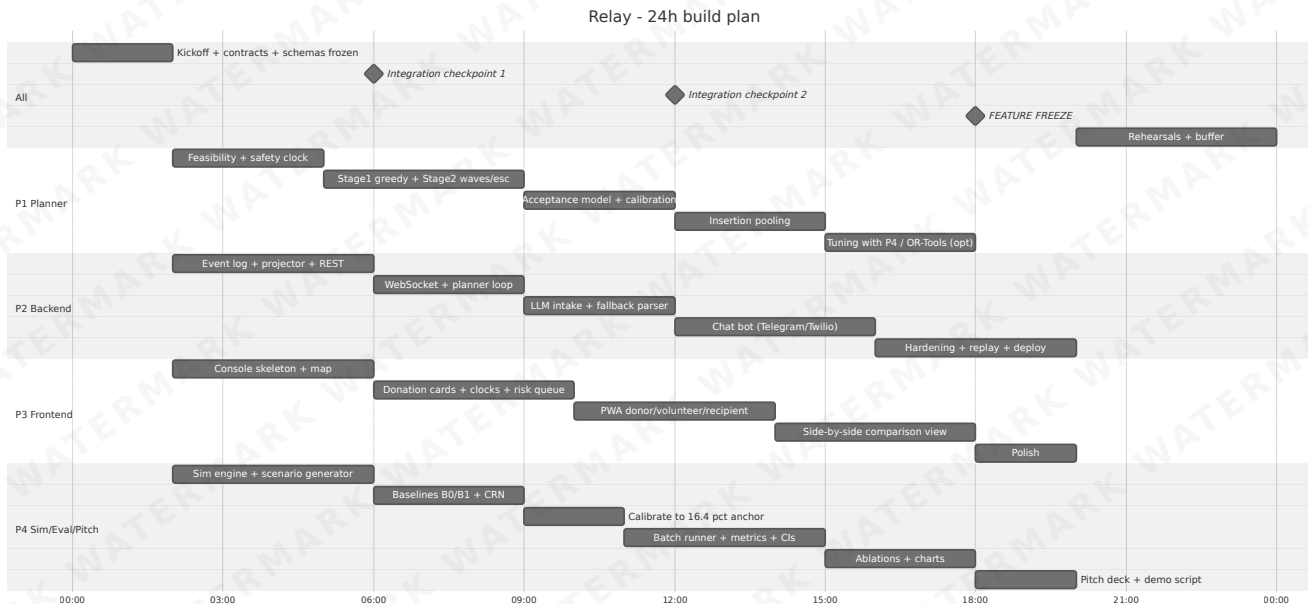
DO NOT BUILD

- RL of any kind; an agentic LLM negotiator; CV quantity estimation; demand forecasting models
- User-account systems beyond simple role links/OTP; payments; blockchain
- Microservices, Kafka, Kubernetes, PostGIS
- A native mobile app

12.2 Team split (4 people)

Role	Owns	Hand-off contract
P1: Planner/OR	<code>planner/</code> (feasibility, stage 1–3, escalation), acceptance model	<code>plan(state, now, cfg) -> list[Action]</code> and the dataclasses in <code>core/models.py</code> (frozen by H2)
P2: Backend	FastAPI, event log, projector, WebSocket, notifier, chat webhook, LLM intake	Event schema (frozen by H2), REST/WS contract
P3: Frontend	Console, PWA views, comparison view, map	Consumes the WS diff format + REST
P4: Sim/Eval + Pitch	Simulator, scenarios, baselines, batch runner, metrics, charts, pitch deck, demo script	Uses the planner and the <code>Policy</code> interface; produces <code>results.json</code>

12.3 Timeline



Critical path: the models/event schema (H2) → planner v1 (H9) → sim+planner integration (H12) → comparison view (H18). **Write greedy first:** it is the baseline, the fallback and the thing you demo if OR-Tools never works.

Checkpoint rules:

- **H6:** a donation posted via the form shows up on the console with a safety clock.
- **H12:** the `friday_night` sim runs end-to-end for B1 and Relay with the same seed.
- **H18:** feature freeze. After this, only bug fixes and demo polish.
- **H20:** full dry run. **Record a backup video of the demo.**

13. Demo Strategy

13.1 Scenario: "Friday, 9:30 pm, one city"

Start with 12 recipients and 45 volunteers from real OSM locations in one city. The simulation runs at 60× (1 real second is 1 simulated minute), with pause and step controls.

13.2 Script (about 3 minutes of live demo)

t (real)	What happens	What the audience sees
0:00	Presenter pastes a WhatsApp-style message: <i>"30 plates veg biryani + 40 rotis, kept hot, we close 11"</i>	LLM extracts a card in about 2 s (category: cooked-hot, veg, ~10 kg, ~30 meals, ready until 23:00). Safety clock 4 h starts. One tap to confirm.
0:20	The planner picks shelter "B" over the nearer "A"	Tooltip: <i>"A is veg-OK but its next service is 7 am with no cold storage → food would be unusable (μ_r). B serves at 10:30 pm."</i> This shows domain depth.
0:35	Wave 1: 2 volunteers	P(claim) gauge at 0.71. Slack 62 min. L_d marker on the timeline.

t (real)	What happens	What the audience sees
0:50	Disruption burst: volunteer Ravi (who accepted) cancels; shelter B marks "full for cooked"; a caterer posts 25 kg	Three cards flash. The planner re-plans (measured latency shown on screen; target under 100 ms): recipient switches to C, wave 2 widens to 4 people, the caterer's food is offered as a +7 min pooled insertion to a volunteer already en route.
1:30	A suburb bakery donation has only 2 eligible volunteers $\rightarrow P^{\max} = 0.48$	The coordinator alert fires immediately , 1 h 40 min before L_d , with a one-click "call Meena (regular)" action.
2:00	Split screen: same seed, baseline B1	On the left, B1's biryani is still unclaimed when the restaurant closes at 23:00 and gets binned (a red card). On the right, Relay delivers it. The counters diverge.
2:30	Results panel: 30 seeds $\times$ friday_night	A table of rescue rate, notifications/rescue, late escalations, zone gap, with 95% CIs labelled "simulated" .
2:50	The donor's phone gets an impact receipt	"Delivered to C at 22:41 · 10 kg · ~30 meals"

13.3 Demo rules

- **Pre-seed everything.** Live chat is a bonus; a local fallback button injects the same message.
- **Fix the seed** so the story plays out the same way on stage. **Then** show the 30-seed table, so the audience knows the story isn't cherry-picked.
- **Show one moment where Relay is *not* better.** For example, a quiet-night seed where B1 and Relay tie. This builds credibility.
- Keep a **backup recording** ready.

14. The Judges' Perspective

No official rubric is given in the PS. What follows is our analysis, not official criteria.

14.1 Likely to impress

- A clear **"real problem" insight** backed by a real statistic (16.4% of rescues needed rescuing [R1]) rather than generic food-waste numbers.
- A **baseline-versus-system comparison on the same seed**, with CIs and honest labelling.
- **Domain depth:** the use-lead (μ_r), food-safety law, confidential shelters, notification fatigue.
- **AI restraint:** explaining why RL is *not* used, with evidence, and where LLMs *are* used, narrowly.
- The PS says "a well-executed matching engine beats a bolted-on AI feature." Relay is literally that.

14.2 Likely to raise scepticism

- "All your results are simulated." $\rightarrow$ Answer in §18.3. We calibrated to one published anchor, and we name what would validate it in the field.
- "Your acceptance model is trained on data you generated." $\rightarrow$ Yes. The pipeline is real; its predictive power is not claimed. The escalation deadline doesn't depend on it.
- "Volunteers in India at 11 pm?" $\rightarrow$ That's a real product risk (§19). The early-escalation design exists *because* supply may be thin.

- "Why not just call people?" → Relay tells the coordinator *whom* and *when*, and does it earlier than the last hour.

14.3 What makes a project look generic

A marketing landing page; an "AI" label on a nearest-neighbour match; a dashboard of fake impact numbers; no baseline; features listed but not demonstrated; a chatbot that only says hello.

14.4 What makes it memorable

The split-screen moment when the baseline's biryani card turns red while Relay's is delivered. And the line: **"You can't dispatch a volunteer. You can only ask. Relay decides who to ask, how many, and when to stop asking and pick up the phone."**

15. Failure & Adversarial Analysis

#	Failure mode	Effect	Mitigation / fallback
1	p model badly wrong	Waves too small (late claims) or too large (spam)	L_d does not depend on p, so escalation still fires in time; online Beta updates correct per-volunteer bias; ablation shows the degradation curve
2	Correlated non-response (rain, festival, late night)	Noisy-OR overestimates P	Context multipliers; coordinator "low-supply" toggle; $P^{\max}$ rule escalates earlier when supply is thin
3	Missing or ambiguous category	Wrong safety horizon	Default to the shortest horizon; confirm card; pickup checklist can downgrade
4	Donor lies or is wrong about hot-holding	Food unsafe	Clock starts at posting unless holding is confirmed; volunteer condition check; "divert" path; donor reliability score (cf. [R21])
5	Recipient capacity stale	Delivery refused	Soft-hold confirmation for large drops; staleness flag after N hours; fallback recipient precomputed
6	Volunteer cancels after accepting	Rescue at risk	Event-triggered re-plan; the state machine returns to Offered; the wave widens automatically because slack shrank
7	Multiple simultaneous disruptions	Plan thrash	Event batching (debounce 2 s); hysteresis on recipient changes; the risk queue orders human attention by slack
8	All volunteers unavailable	Nothing gets claimed	Escalation at posting ($P^{\max} \approx 0$); backup courier; honest message to the donor ("we can't make it tonight") instead of silence
9	Optimiser fails or times out	No plan	The greedy plan is always computed first; OR-Tools is only an upgrade
10	LLM hallucination in intake	Wrong quantity or category	Schema validation, confirm card, conservative defaults, regex cross-check (for example that

#	Failure mode	Effect	Mitigation / fallback
			numbers in the text match the extracted quantity)
11	Notification spam	Volunteer churn	Budget K per week, fatigue discount γ , deadline-sized waves
12	Equity drift (downtown gets everything)	Periphery unserved	Fill-rate term; per-zone metrics; the early escalation rule
13	Privacy breach (shelter address)	Harm to vulnerable people	Confidential flag, proxy handover, location minimisation
14	Gaming the impact numbers	Loss of credibility	Event-sourced ledger; meals only on <i>Delivered</i> ; ReFED-style conservative adjustments noted [R9]
15	Scale (10× load)	Latency	Zone partitioning; stage 1–2 are near-linear; OR-Tools windowed to the most urgent N
16	Wrong assumptions (for example, that ad-hoc rescues dominate)	Wrong product	Recurring-rescue conversion is built in; the pilot measures the ad-hoc versus scheduled share first
17	ML performs worse than the constant-p baseline	–	Ship constant p (the ablation tells us); the system still works. This is designed in, not bolted on.

16. Business & Real-World Deployment

16.1 Who adopts, and why

- **Adopter:** a food-rescue org or chapter (for example an IFSA member NGO [R18], or a volunteer chapter model like Robin Hood Army [R16]). The coordinator gets back hours of time every night.
- **Anchor donor for a pilot [PROP]: campus dining at the team's own university.** It is predictable, recurring, has a single contact, and yields clean data. It also supplies the first real offer/accept logs the acceptance model needs.
- **Participants** use chat and SMS, so there is no install barrier.

16.2 Who pays? Honest options

Model	Precedent	Fit in India	Verdict
NGO SaaS licence	Food Rescue Hero licenses its platform to other rescue orgs [R12]	Low willingness to pay	Possible at low price; not a business on its own
Donor pays for logistics + compliance/ESG reports	Copia charges donors a % of donation value; SB 1383 creates a mandate in CA [R4, R11]	No equivalent Indian mandate that we found; FSSAI requires records [R6] but not recovery quotas	Weaker in India; viable for hotel chains and corporate cafeterias with ESG reporting needs

Model	Precedent	Fit in India	Verdict
CSR-funded deployments	Schedule VII item (i) covers hunger [R20]	Strong: companies must spend CSR money and want auditable outcomes	Recommended primary path: a CSR sponsor funds city deployments, and Relay's ledger provides the auditable outcome report
Government / regulator partnership	FSSAI Save Food Share Food + IFSA (85 NGOs) [R18]	Distribution network already exists	Pursue as a channel, not revenue

Recommendation [PROP]: position Relay as **open-core rescue infrastructure**. The core is free for NGOs and CSR-funded per city. Paid multi-site reporting goes to enterprise donors (hotel groups, corporate campuses). Don't lead the pitch with a business model; lead with the reliability insight, and have this table ready for questions.

16.3 Unit costs (order-of-magnitude, to be confirmed)

Per rescue: roughly 3–6 chat messages, one short LLM call (intake), and negligible compute. The main recurring costs are **messaging fees** (WhatsApp business messaging is billed per message or template category; check Meta/Twilio pricing at build time) and **backup courier trips** when volunteers are unavailable. That is why escalation is counted as a cost in §9.7.

16.4 Operational constraints

- Volunteer food-safety orientation and insulated carriers. Donation agencies must meet FSSAI hygiene, transport and storage obligations [R6].
- Recipient onboarding: receiving hours, storage, dietary rules and use lead μ_r . That takes a 10-minute setup call per recipient.
- A local health authority should validate the category→horizon table (§Appendix A) before live use.

16.5 Pilot plan (post-hackathon, 6 weeks)

1. **Weeks 1–2, shadow mode:** the coordinator runs WhatsApp as usual while Relay ingests the same posts and makes recommendations only. Measure the **ad-hoc versus scheduled share**, night-time response rates and recipient hour coverage. *Kill criterion: if fewer than 15% of rescues are ad-hoc, pivot toward recurring-schedule tooling.*
2. **Weeks 3–6, live:** Relay sends the offers, and the coordinator handles escalations. Compare rescue rate, notifications per rescue and coordinator minutes per night against weeks 1–2.
3. Fit the acceptance model on **real** logs, and report calibration honestly.

17. Implementation Blueprint

17.1 Repository layout

```

relay/
├── README.md           # run instructions, demo script
├── pyproject.toml       # fastapi, uvicorn, pydantic, ortools, scikit-learn, numpy, httpx
├── config/
│   ├── safety_rules.yaml # category -> horizon, conservative defaults (Appendix A)
│   └── planner.yaml      # tau*, Δ, K, γ, ε ... (Appendix B)

```

```

├── scenarios/                # friday_night.yaml, surge.yaml, sparse_suburb.yaml ...
├── core/
│   ├── models.py            # frozen dataclasses: Donation, Recipient, Volunteer, Offer, Assignment, State
│   ├── events.py            # event types + payload schemas
│   ├── projector.py         # fold(events) -> State
│   ├── geo.py               # haversine, cached OSRM matrix
│   └── safety.py            # safe_until(), feasible()
├── planner/
│   ├── __init__.py          # plan(state, now, cfg) -> list[Action]
│   ├── stage1_recipient.py
│   ├── stage2_offers.py     # waves, L_d, escalation
│   ├── stage3_insertion.py
│   ├── ortools_pdptw.py     # optional
│   ├── acceptance.py        # logistic + Beta shrinkage; fit/predict
│   └── baselines.py         # B0 broadcast, B1 static tiered, B2 optimiser-only
├── api/
│   ├── main.py              # FastAPI app, REST, WS
│   ├── intake.py            # LLM extraction + regex fallback
│   ├── notifier.py          # telegram / twilio / console
│   └── store.py             # SQLite event log
├── sim/
│   ├── engine.py            # discrete-event loop
│   ├── world.py             # CRN pre-generation per seed
│   ├── agents.py            # volunteer/recipient/donor behaviour
│   ├── runner.py            # scenario x policy x seeds -> results.json
│   └── metrics.py           # $11.2 metrics + bootstrap CIs
├── web/                      # React + Vite + Leaflet
│   └── src/pages/{Console,Compare,Donor,Volunteer,Recipient,Impact}.tsx
├── data/
│   ├── osm_city.geojson     # fetched once via Overpass
│   └── osrm_matrix.npz      # cached
└── tests/
    ├── test_safety.py       # no plan ever violates s_d
    ├── test_planner_props.py # property tests (below)
    └── test_replay.py       # replay(events) deterministic

```

17.2 Core planner (pseudocode close to real code)

```

def plan(state: State, now: datetime, cfg: Cfg) -> list[Action]:
    acts: list[Action] = []
    open_ds = [d for d in state.donations if d.status in OPEN]

    # Stage 0: hard safety. Nothing past its deadline is ever routed.
    for d in open_ds:
        if now >= d.safe_until - cfg.eps_safety:
            acts.append(Expire(d.id))
    open_ds = [d for d in open_ds if not any(a.target == d.id for a in acts)]

    # Stage 1: recipient selection, most urgent first
    cap = state.capacity_snapshot(now)
    for d in sorted(open_ds, key=lambda d: robust_slack(d, state, now)):
        if d.recipient and still_feasible(d, d.recipient, state, now):
            if not better_by(d, d.recipient, state, cfg.delta): # hysteresis: switch only for a big gain
                continue
        cand = [r for r in state.recipients if feasible_recipient(d, r, cap, state, now, cfg)]
        if not cand:

```

```

    acts.append(Escalate(d.id, reason="no_feasible_recipient")); continue
    r = max(cands, key=lambda r: score(d, r, state, cfg))
    cap.reserve(r, d)
    acts.append(ChooseRecipient(d.id, r.id))

# Stage 2: deadline-aware waves + escalation
for d in unclaimed(open_ds, acts):
    r = recipient_of(d, acts, state)
    L = latest_safe_escalation(d, r, state.backup, cfg)          # §9.4, rules + travel only
    E = [v for v in state.volunteers
          if v.available(now) and v.offers_7d < cfg.K
          and feasible_triple(v, d, r, state, now, cfg) and not state.offered(v, d)]
    O = state.outstanding_offers(d)
    p = {v.id: acc_model.p(v, d, now) for v in E + O}
    pmax = 1 - prod(1 - p[v.id] for v in E + O)

    if not d.escalated and (now >= L - cfg.alert_lead or pmax < cfg.tau_min):
        acts.append(Escalate(d.id, L=L, pmax=pmax))          # early, explainable, idempotent
    if state.wave_pending(d, now):
        continue          # wait for timeout
    W = floor((L - now) / cfg.wave_timeout)
    if W < 1:
        acts.append(SendOffers(d.id, [v.id for v in E]))      # blast what's left
        continue
    tau_w = 1 - (1 - cfg.tau_star) ** (1 / W)
    ranked = sorted(E, key=lambda v: p[v.id] / (1 + cfg.gamma * v.offers_7d), reverse=True)
    wave, miss = [], prod(1 - p[v.id] for v in O)
    for v in ranked:
        if 1 - miss >= tau_w or len(wave) >= cfg.k_max: break
        wave.append(v.id); miss *= (1 - p[v.id])
    if wave:
        acts.append(SendOffers(d.id, wave, expires=now + cfg.wave_timeout))

# Stage 3: pooling offers onto active routes
for d in unclaimed(open_ds, acts):
    best = cheapest_feasible_insertion(d, state.active_routes, cfg.detour_max)
    if best:
        acts.append(SendOffers(d.id, [best.volunteer], kind="insertion", detour=best.delta))
return dedupe(acts)

```

Property tests (must pass):

1. For any random state, no `ChooseRecipient` / `SendOffers` produces a triple violating §9.2. (Hypothesis-style fuzzing.)
2. `plan` is deterministic given `(state, now, cfg)`.
3. Replaying the event log gives the same state as the live projector.
4. No volunteer receives more than `K` offers in any 7-day window.
5. Accepted assignments are never reassigned without a `ClaimCancelled` event.

17.3 Simulator loop

```

def run(scenario, policy, seed) -> Metrics:
    world = World.generate(scenario, seed)          # CRN: arrivals, shifts, U[v,d] accept draws, cancels
    q = EventQueue(world.exogenous_events())        # DonationPosted, ShiftStart, CapacityDrop, ...
    state, log = State.empty(world), []
    while q and q.peek().t <= world.horizon:
        ev = q.pop(); log.append(ev); state = project(state, ev)
        for act in policy(state, ev.t, cfg):        # SAME planner function as prod
            log.append(act); state = project(state, act)
            q.extend(world.respond(act, state))      # e.g., OfferSent -> accept at t+delay if U[v,d] < p_true

```

```
q.push_tick(ev.t, every=60)
return metrics.compute(log, world)
```

17.4 LLM intake contract

System prompt (condensed):

Extract a food donation from the message. Output **only** JSON matching the schema. Never guess safety facts: if the holding temperature isn't stated, set `holding` to `"unknown"`. If quantity is ambiguous, give `qty` as `null` and keep the raw phrase in `notes`. Don't invent items.

```
{
  "items": [{"name": "string", "qty": "number|null", "unit": "plates|kg|pieces|trays|boxes|litres|null"}],
  "category": "cooked|bakery|produce|dairy|packaged|mixed|unknown",
  "holding": "hot|cold|ambient|unknown",
  "veg": "true|false|null",
  "ready_until": "HH:MM|null",
  "weight_kg_est": "number|null",
  "notes": "string"
}
```

Post-processing:

- Validate against the schema.
- Cross-check that the numbers in the raw text appear among the extracted quantities.
- `unknown` holding means the clock starts at posting time.
- Missing weight is estimated from a units table (for example 1 plate $\approx$ 0.35 kg [ASM]).
- The donor confirms or edits.
- **Timeout 3 s $\rightarrow$ form fallback.**

17.5 Frontend screens

Screen	Key elements
Coordinator Console	Map (donors, recipients, volunteers, routes); risk queue sorted by slack with safety-clock countdowns; per-donation panel (recipient + reason, $P(\text{claim})$ gauge, L_d marker, wave history); escalation inbox with one-click actions; recipient capacity board
Compare	Same-seed split screen B1 versus Relay, synced sim clock; results table (30 seeds, CIs, "simulated" label); ablation bars
Donor (PWA/chat)	Paste or type $\rightarrow$ confirm card $\rightarrow$ honest ETA / "can't make it" $\rightarrow$ receipt
Volunteer (PWA/chat)	Offer card (area, distance, food, deadline, detour) $\rightarrow$ accept/decline $\rightarrow$ navigation link $\rightarrow$ pickup checklist + photo $\rightarrow$ handover code entry
Recipient (PWA/chat)	Tonight's capacity toggles per storage class; veg-only; expected arrivals; code display
Impact	kg, meals, deliveries, per-recipient, per-zone; method box (meal factor, CO2e factor and source); CSV export

17.6 Deployment for demo day

- Primary: API on one small cloud VM plus the frontend on a static host.
- **Fallback: everything on one laptop** (uvicorn + vite preview), with the sim needing no internet.
- Pre-cache OSM tiles for the demo area, or use a simple vector basemap.
- Keep a demo recording as the last-resort fallback.

18. Pitch & Demo

18.1 A 5-minute structure

Time	Segment	Content
0:00–0:25	Hook	"At 10:45 on a Friday night, a restaurant in this city has 30 plates of biryani and 4 hours before it becomes unsafe. The shelter wants it. A volunteer lives 2 km away. It still goes in the bin. Not because nobody cares, but because nobody <i>commits</i> in time."
0:25–1:00	Insight	"We thought this was a matching problem. It isn't. Matching platforms report a 99% match rate [R4]. In the best public dataset on food rescue, 412 Food Rescue in Pittsburgh, one in six rescues went unclaimed or needed a dispatcher to phone people in the final hour [R1]. You can't dispatch a volunteer. You can only ask. "
1:00–1:30	Product	"Relay is a co-pilot for the rescue coordinator. For every donation it keeps a live answer to: <i>will this food make it, and if not, when must a human step in?</i> "
1:30–3:30	Live demo	§13.2 script
3:30–4:10	How it works	Three clocks → slack; deadline-sized offer waves ($\tau_w = 1 - (1 - \tau^*)^{1/W}$); escalation time L_d that doesn't depend on ML; the same planner running production and the simulator
4:10–4:35	Honesty + results	"These results are simulated, calibrated to the published 16.4%. We didn't use RL, because the evidence in this exact domain has deep RL losing to greedy [R2]. The LLM only reads messages; it never makes safety calls."
4:35–5:00	Close	"Pilot plan: our campus dining hall plus one NGO chapter, six weeks, shadow mode first. Relay makes every donation someone's job, before the clock runs out. "

18.2 Talking points to repeat

1. "The bottleneck is commitment, not matching."
2. "Three clocks: food safety, donor, recipient. The shelter's next meal time matters as much as expiry."
3. "Notifications are a budget, not a free resource."
4. "The ML can be wrong; the escalation deadline can't be, because it's physics and rules."
5. "Same seed, same world, different policy."

18.3 Hard judge questions and honest answers

Question	Answer
"All your results are simulated. Why should we believe them?"	"You shouldn't treat them as real-world numbers, and we don't. They show that the <i>mechanism</i> works under stated assumptions, and we calibrated the baseline to the one published statistic. The pilot has a shadow-mode phase to measure exactly this."

Question	Answer
"Your ML is trained on your own synthetic data."	"Correct. It's a pipeline demo. The design point is that its errors can only move the alert earlier or later <i>within</i> the escalation deadline, never past it. Our ablation with a deliberately wrong model shows the degradation."
"Why not reinforcement learning?"	"No real environment data, and in the published food-rescue notification benchmark DQN scored below greedy [R2]. The upgrade path is contextual bandits once real logs exist."
"Isn't this just Uber for food?"	"Uber can dispatch drivers who are paid to accept. We can't. And Uber's payload doesn't become unsafe at a fixed time or need a receiver whose kitchen is open. The core of Relay is managing <i>uncertain consent</i> under a hard deadline."
"How is this different from 412 Food Rescue / Food Rescue Hero?"	"They are the leader, and their published research shows the problem we attack. Their documented 2018–19 notification scheme was static for every rescue [R1]. Ours is per-donation and deadline-driven. They may have evolved since, so we claim a difference from the <i>published</i> approach, not uniqueness."
"What about food safety liability?"	"Safety is rule-based, not ML, using FDA time-as-a-public-health-control limits as conservative defaults [R5], configurable for FSSAI guidance. The clock starts at posting unless holding is confirmed. There is a pickup condition check, a divert path and a full audit log, which the 2019 FSSAI regulations require [R6]."
"Why would a restaurant bother?"	"Thirty seconds by chat, an honest ETA, and a receipt. For corporate donors, auditable CSR/ESG reporting. We don't claim Indian tax benefits, because we couldn't verify them."
"Will volunteers exist at 11 pm in India?"	"That's our biggest product risk. Relay's early-escalation rule is designed for thin supply: it tells the coordinator at posting time, not in the last hour. The pilot measures night-time response rates first."
"What if two big things happen at once?"	"The demo does exactly that: a cancellation, a full shelter and a surge within a minute. Events are debounced, re-planning targets under 100 ms (we show the measured latency), and hysteresis stops recipients flip-flopping."
"Does multi-stop routing actually matter?"	"Less than people think for volunteer cars. Our ablation measures it. It matters for surges and big caterer loads, which is why it's an insertion <i>offer</i> , not the core."
"How does this scale?"	"Rescue is local, so we partition by zone, and the planner is stateless over an event log. Stages 1–2 are near-linear. OR-Tools is optional and time-boxed."
"Where's the CO2 number from?"	"Meals use the 1.2 lb/meal convention [R32]. For CO2e we show the factor and its source on screen. Factors vary widely by food type, so we don't put a single headline CO2e figure on a slide without its method."
"What happens if the LLM misreads 'forty' as '14'?"	"Schema validation, a numeric cross-check against the raw text, a donor confirm card, and a form fallback. The LLM never touches safety fields except to say 'unknown'."

19. Final Critical Verdict

Strong

- The problem reframing (commitment and reliability rather than matching) rests on real published data [R1]. It gives the whole project a thesis instead of a feature list.
- The deadline-aware wave rule and the ML-independent escalation deadline are simple, explainable and testable. They extend a documented real-world practice [R1] with a formula taken from the peer-reviewed literature [R2].
- The shared planner for production and simulation plus the same-seed comparison is methodologically sound and makes a compelling demo.
- The team knows where AI helps (extraction) and where it doesn't (safety, RL).

Weak

- **All quantitative evidence is simulated.** The acceptance model's predictive value is unproven. A sceptical judge can discount the results table heavily, and that's fair.
- Much of the difference from incumbents rests on *published* descriptions that may be out of date.
- The independence assumption in noisy-OR is optimistic on correlated nights.

Unnecessarily complicated (cut if time is short, in this order)

1. OR-Tools PDPTW. Insertion will probably match it at this scale.
2. A real WhatsApp/Telegram integration. A web chat box demonstrates the same thing.
3. OSRM. Haversine $\times 1.35$ is fine for a demo.
4. CI bootstrap machinery. Report mean $\pm$ SE across seeds if time is short.

Actually novel (for a hackathon, and plausibly in practice)

- A per-donation, deadline-derived notification schedule $\tau_w = 1 - (1 - \tau^*)^{1/W_d}$ combined with a model-independent latest-safe-escalation time and the $P^{\max}$ early-escalation trigger.
- A safety deadline defined at *use* (μ_r), not at delivery.

Not novel

- Rolling-horizon dispatch, PDPTW, noisy-OR claim probability, Beta shrinkage, LLM extraction and event sourcing. All are standard. Say so; the novelty is in how they're put together for this problem.

Biggest technical risk: integrating live system, planner, simulator and comparison UI in 24 hours. *Mitigation:* freeze the models at H2, build greedy first, and set the H12 end-to-end checkpoint.

Biggest product risk: the assumption that ad-hoc, late-evening rescues are a large enough share, and that volunteer supply exists then. Food Rescue US's emphasis on "adopted" recurring rescues [R13] hints that mature operations turn most volume into schedules. *Mitigation:* recurring-rescue conversion is a built-in feature, and the pilot's kill criterion tests this.

What could make the project fail on stage: a UI-heavy demo where the comparison isn't visible, or a live-network failure. Or overclaiming. One inflated number ("we cut food waste 40%") undoes the credibility that the rest of the project earns.

The one thing to execute exceptionally well: the same-seed split-screen disruption moment. The baseline's card goes red while Relay escalates early and re-routes, followed immediately by the 30-seed table labelled "simulated." If that 60 seconds works, the project wins the argument.

Appendix A: Safety rules table (conservative defaults; must be validated locally) [R5, ASM]

Category	Holding	Horizon H	Notes
Cooked (TCS)	hot $\geq 57^{\circ}\text{C}$ / 135°F at removal	4 h	FDA TPHC; discard at 4 h
Cooked (TCS)	cold $\leq 5^{\circ}\text{C}$ / 41°F at removal	4 h, or 6 h if $\leq 21^{\circ}\text{C}$ / 70°F throughout	FDA TPHC 6-hour cold option
Cooked	unknown	4 h from posting	Conservative default
Dairy / cut fruit / salads	cold	4 h	Treat as TCS
Bakery (non-cream)	ambient	24 h [ASM]	Cream/custard items $\rightarrow$ TCS
Whole produce	ambient	48 h [ASM]	Visual check at pickup
Packaged	sealed	until label date [ASM]	Must not be past date

Appendix B: Planner defaults (tune in simulation)

Param	Default	Meaning
τ^*	0.95	Target $P(\text{claimed before } L_d)$
$\tau_{\min}^*$	0.80	Early-escalation threshold on $P^{\max}$
Δ	8 min	Wave timeout
$k_{\max}$	6	Max volunteers per wave
K	10 / week	Offer budget per volunteer
γ	0.15	Fatigue discount
ε	20 min	Safety buffer
$\varepsilon_{\text{esc}}, \Delta_{\text{alert}}$	10 min, 10 min	Escalation buffers
ρ_B	20 min	Backup response time
θ_{detour}	10 min	Max insertion detour
$\lambda_F, \lambda_T, \lambda_S$	0.5, 0.02/min, 0.05/min	Stage-1 weights (meal-normalised)

Appendix C: Assumptions register

ID	Assumption	Where used	How to validate
A1	FDA TPHC horizons are acceptable defaults in India	Safety clock	Local food-safety authority / FSSAI guidance

ID	Assumption	Where used	How to validate
A2	Volunteer responses are independent within a wave	Noisy-OR	Real logs; the correlated-rain stress test
A3	Evening surplus peak; recipients mostly close about 21:00	Simulator	NGO interviews; shadow-mode pilot
A4	8% cancel-after-accept; 4-min median response	Simulator	Pilot logs
A5	1 plate $\approx$ 0.35 kg; 1 meal = 1.2 lb	Impact	Weigh samples; state the convention
A6	A backup resource exists, responding in 20 min	L_d	Org's actual on-call practice
A7	Ad-hoc rescues are a meaningful share	Product thesis	Pilot kill criterion (§16.5)

References

All sources were accessed in September 2026. Where we could read only an abstract, a listing or a secondary description, the report says so.

- [R1] Z. R. Shi, Y. Yuan, K. Lo, L. Lizarondo, F. Fang. "Improving Efficiency of Volunteer-Based Food Rescue Operations." *IAAI-20 / AAAI 2020*, pp. 13369–13375. <https://cdn.aaai.org/ojs/7051/7051-13-10280-1-10-20200526.pdf> · <https://ojs.aaai.org/index.php/AAAI/article/view/7051>
- [R2] N. Raman, Z. R. Shi, F. Fang. "Global Rewards in Restless Multi-Armed Bandits." *NeurIPS 2024*. <https://arxiv.org/abs/2406.00738> · https://proceedings.neurips.cc/paper_files/paper/2024/hash/2bf9868e940198406713451d7656c981-Abstract-Conference.html · code: <https://github.com/naveenr414/food-rescue-rmab>
- [R3] A. Tang, N. Raman, F. Fang, Z. R. Shi. "Contextual Budget Bandit for Food Rescue Volunteer Engagement." arXiv preprint, 2025. <https://arxiv.org/abs/2509.10777> (The ~90% vs ~40% completion figure is this paper's description of Shi et al., WWW 2021, <https://dl.acm.org/doi/10.1145/3442381.3449787>, which we could not access directly.)
- [R4] Copia. Solutions and FAQ pages. <https://gocopia.com/solutions/> · <https://gocopia.com/faq-support/>
- [R5] FDA Food Code §3-501.19, Time as a Public Health Control, as summarised by NH DHHS. <https://www.dhhs.nh.gov/sites/g/files/ehbemt476/files/documents/2021-11/fp-tphc.pdf>
- [R6] Food Safety and Standards (Recovery and Distribution of Surplus Food) Regulations, 2019, summarised by S.S. Rana & Co. <https://ssrana.in/articles/fssai-food-donation-regulations/>
- [R7] UNEP. "World squanders over 1 billion meals a day." Press release on the Food Waste Index Report 2024. <https://www.unep.org/news-and-stories/press-release/world-squanders-over-1-billion-meals-day-un-report> · Report: <https://www.unep.org/resources/publication/food-waste-index-report-2024>
- [R8] WRI India. "Defining Food Waste for Sustainable Food Systems" (Sept 2024). <https://wri-india.org/perspectives/defining-food-waste-sustainable-food-systems>
- [R9] ReFED. The Problem page; Glossary; Impact Calculator Methodology (Sept 2024). <https://refed.org/food-waste/the-problem/> · <https://docs.refed.org/glossary.html> · <https://insights-engine.refed.org/impact-calculator>
- [R10] US EPA. "Quantifying Methane Emissions from Landfilled Food Waste" (2023). <https://www.epa.gov/land-research/quantifying-methane-emissions-landfilled-food-waste> · also "From Farm to Kitchen" (2021): <https://www.epa.gov/land-research/farm-kitchen-environmental-impacts-us-food-waste>
- [R11] CalRecycle. SB 1383 Food Recovery. <https://calrecycle.ca.gov/organics/slcp/foodrecovery/> · <https://calrecycle.ca.gov/organics/slcp/foodrecovery/donors/>

- [R12] Food Rescue Hero. <https://foodrescuehero.org/> · Wikipedia, "412 Food Rescue": https://en.wikipedia.org/wiki/412_Food_Rescue
- [R13] Food Rescue US. Our App; FAQs. <https://foodrescue.us/our-app/> · <https://foodrescue.us/about/faqs/>
- [R14] Too Good To Go. About us. <https://www.toogoodtogo.com/en-us/about-us>
- [R15] OLIO. Our impact. <https://olioapp.com/en/our-impact/>
- [R16] Robin Hood Army. <https://robinhoodarmy.com/> · https://en.wikipedia.org/wiki/Robin_Hood_Army
- [R17] Feeding India. <https://www.feedingindia.org/>
- [R18] Nutrition Connect. "FSSAI guiding the optimisation of surplus food donation" (2023). <https://nutritionconnect.org/food-safety-standards-authority-india-fssai-guiding-optimisation-surplus-food-donation>
- [R19] Food Donation Improvement Act of 2022, Public Law 117-362. <https://www.govinfo.gov/content/pkg/PLAW-117publ362/html/PLAW-117publ362.htm>
- [R20] Companies Act 2013, Schedule VII. <https://ca2013.com/schedule/schedule7-3/>
- [R21] Raman et al. "RescueLens: LLM-Powered Triage and Action on Volunteer Feedback for Food Rescue." arXiv 2025 / AI Magazine 2026. <https://arxiv.org/abs/2511.15698> · <https://onlinelibrary.wiley.com/doi/full/10.1002/aaai.70080>
- [R22] D. J. Nair, D. Rey, V. V. Dixit. "Fair allocation and cost-effective routing models for food rescue and redistribution." *IIE Transactions* 49(12):1172–1188, 2017. <https://www.tandfonline.com/doi/full/10.1080/24725854.2017.1351043> (abstract-level only)
- [R23] R. W. Lien, S. M. R. Iravani, K. R. Smilowitz. "Sequential Resource Allocation for Nonprofit Operations." *Operations Research* 62(2):301–317, 2014. <https://doi.org/10.1287/opre.2013.1244>
- [R24] A. Mittal, N. Gibson, C. Krejci et al. "An Agent-based Model of Surplus Food Rescue Using Crowd-shipping." *Winter Simulation Conference 2019*. <https://ieeexplore.ieee.org/document/9004732/> · "Crowd-shipping for urban food rescue logistics." *IJPDLM* 51(5), 2021. <https://www.emerald.com/ijpdlm/article-abstract/51/5/486/161619/> (listing-level only)
- [R25] J. Alonso-Mora, S. Samaranayake, A. Wallar, E. Frazzoli, D. Rus. "On-demand high-capacity ride-sharing via dynamic trip-vehicle assignment." *PNAS* 114(3):462–467, 2017. <https://www.pnas.org/doi/10.1073/pnas.1611675114>
- [R26] Z. Xu et al. "Large-Scale Order Dispatch in On-Demand Ride-Hailing Platforms: A Learning and Planning Approach." *KDD 2018*, pp. 905–913. <https://doi.org/10.1145/3219819.3219824>
- [R27] Google OR-Tools: Pickup & Delivery, Penalties/Disjunctions, VRPTW; repo (Apache-2.0). https://developers.google.com/optimization/routing/pickup_delivery · <https://developers.google.com/optimization/routing/penalties> · <https://developers.google.com/optimization/routing/vrptw> · <https://github.com/google/or-tools>
- [R28] VROOM. <https://github.com/VROOM-Project/vroom> · API: <https://github.com/VROOM-Project/vroom/blob/master/docs/API.md>
- [R29] OSRM. <https://project-osrm.org/> · Demo-server policy: <https://github.com/Project-OSRM/osrm-backend/wiki/Demo-server>
- [R30] SINTEF TOP. Li & Lim PDPTW benchmark. <https://www.sintef.no/projectweb/top/pdptw/li-lim-benchmark/>
- [R31] Twilio. Test WhatsApp messaging with the Sandbox. <https://www.twilio.com/docs/whatsapp/sandbox>
- [R32] Feeding America. "About our claims" (1.2 lb per meal). <https://www.feedingamerica.org/ways-to-give/faq/about-our-claims>

Could not verify, so not used as facts: the India row of the UNEP FWI 2024 annex (we used WRI India's secondary citation, flagged); Indian tax treatment of in-kind food donations; the internals of proprietary platforms today; any single per-kg CO₂e factor (it depends on food type; pick one from [R9]/[R10] and cite it on screen).